Өмнөговь Технологийн Дээд Сургууль
Мэдээллийн Технологийн Тэнхим

М.Түвшинжаргал

Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт

Бакалаврын төгсөлтийн ажил

Даланзадгад хот

Өмнөговь Технологийн Дээд Сургууль
Мэдээллийн Технологийн Тэнхим

Мэдээллийн технологийн тэнхим

Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт

Мэргэжлийн индекс: D061304
Мэргэжил: Мэдээллийн технологи
Удирдагч: Доктор Д.Эрдэнэтуяа
Зөвлөгч:
Гүйцэтгэгч: М.Түвшинжаргал

Даланзадгад хот
2026 он

Батлав. Мэдээллийн технологийн тэнхимын эрхлэгч:

..... //

Удирдагч: ..... /Доктор Д.Эрдэнэтуяа/

ТӨГСӨЛТИЙН АЖЛЫН ТӨЛӨВЛӨГӨӨ

Төгсөлтийн ажлын сэдэв:
Монгол: Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт
Англи: Design and Implementation of a Three-Tier Sports Hall Booking System Based on Cloud Environment

Төслийн зорилго: Өмнөговь аймгийн Даланзадгад хотын спорт заалнуудын цаг захиалгыг цахимжуулах, хэрэглэгчдэд хялбар, хурдан захиалга хийх боломж олгох Flutter мобайл аппликейшн хөгжүүлэх.

Оюутны овог нэр, код: М.Түвшинжаргал/B221940619/

№	Ажлын бүлэг, хэсгийн нэр	эзлэх хувь	дуусах хугацаа
Удиртгал. Оршил ба судалгааны үндэслэл			
1	1.1 Үндэслэл, асуудлын тодорхойлолт 1.2 Зорилго, зорилт 1.3 Судлагдсан байдал, хамрах хүрээ	5%	III.13
Бүлэг №1. Онолын судалгаа			
2	1.1 Мобайл технологи (Flutter, Dart, Firebase, AI чатбот) 1.2 Бэкэнд технологи (Node.js, Express, REST API) 1.3 Үүлэн технологи (Docker, AWS EC2/ECR, GitHub Actions)	30%	IV.10
Бүлэг №2. Програм хангамжийн загварчлал ба хөгжүүлэлт			
3	2.1 Програмын шаардлага ба загварчлал 2.2 Frontend хөгжүүлэлт (Flutter апп, UI/UX) 2.3 Backend хөгжүүлэлт (Node.js REST API, email) 2.4 Өгөгдлийн сан (Firestore collection-уудын загвар)	30%	V.10
Бүлэг №3. Хэрэгжүүлэлт			
4	3.1 Үүлэн байршуулалт (Docker, AWS EC2, CI/CD) 3.2 Туршилт ба үнэлгээний үр дүн	30%	V.22
Ерөнхий дүгнэлт			5%

Төлөвлөгөөг боловсруулсан оюутан: ..... /М.Түвшинжаргал/

ТӨГСӨЛТИЙН АЖЛЫН ЯВЦ

№	Хийж гүйцэтгэсэн ажил	Биелсэн хугацаа	Удирдагчийн гарын үсэг
1	Удиртгал: Оршил, судалгааны үндэслэл боловсруулсан	2026-03-13	
2	Бүлэг 1: Онолын судалгаа хийсэн	2026-04-10	
3	Бүлэг 2: Загварчлал, хөгжүүлэлт хийсэн (Flutter, Node.js)	2026-05-10	
4	Бүлэг 3: Хэрэгжүүлэлт, туршилт хийж, дүгнэлт боловсруулсан	2026-05-22	

Ажлын товч дүгнэлт

.....

.....

.....

.....

.....

.....

.....

Удирдагч: ..... /Доктор Д.Эрдэнэтуяа/

ЗӨВШӨӨРӨЛ

Оюутан М.Түвшинжаргалгын бичсэн төгсөлтийн ажлыг УШК-д хамгаалуулахаар тодорхойлов.

Салбарын эрхлэгч: ..... //

Зохиогчийн эрх хамгаалал

Миний бие М.Түвшинжаргал, Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт сэдэвт энэ ажил нь минийх бөгөөд дараахыг нотолж байна. Үүнд:

- Горилогч энэ ажлыг тус сургуулиас боловсролын зэрэг авахаар бүхэлд нь буюу голлон хийсэн болно.
- Энэ ажлын аль нэг хэсгийг тус сургуульд эсвэл өөр байгууллагад боловсролын зэрэг, мэргэшил авахаар өмнө нь илгээсэн бол түүнийгээ тодорхой заасан болно.
- Бусад хүмүүсийн хэвлүүлсэн ажлаас зөвлөгөө авсан бол түүнийгээ үндэслэсэн болно.
- Бусад хүмүүсийн ажлаас ишлэл хийхдээ эх үүсвэрийг нь заасан болно.
- Миний ажилд тусалсан голлох бүх эх үүсвэрт талархаж байна.
- Ажлыг бусадтай хамтарсан бол алийг нь бусад хүмүүс хийсэн болохыг тодорхой заасан болно.

Гарын үсэг: _____

Огноо: _____

Өмнөговь Технологийн Дээд Сургууль
Мэдээллийн Технологийн Тэнхим

Хураангуй

Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт

М.Түвшинжаргал

Түлхүүр үгс: *Flutter, Firebase, Node.js, Docker, AWS EC2, GitHub Actions, AI чатбот, цаг захиалга, спорт заал, үүлэн орчин, Даланзадгад*

Энэхүү бакалаврын төгсөлтийн ажлын зорилго нь Өмнөговь аймгийн Даланзадгад хотын таван спорт байгууламж — **Өмнөговь аймгийн Спорт цогцолбор** (спорт цогцолбор) болон **Говийн Арена**, **1-р болон 2-р хорооны Спорт заал**, **Даланзадгад Волейболын клуб заал** (хот айлын заалнууд) — -д зориулсан үүлэн орчинд суурилсан цаг захиалгын мобайл аппликейшн хөгжүүлэхэд оршино.

Программ хангамжийн систем нь **Flutter** (Android/iOS), **Node.js/Express** бэкэнд, **Firebase** (Auth, Firestore, Cloud Functions), **Docker/AWS EC2** дэд бүтэц болон **AI чатбот** (Claude Haiku, Groq Llama) гэсэн таван бүрэлдэхүүнтэй гурван давхаргат архитектурт суурилна. Бэкэнд нь **AWS EC2** (ap-southeast-1) дээр Docker контейнераар ажиллаж, **GitHub Actions** CI/CD хоолойгоор автоматаар удирдана.

Үндсэн дэд функцүүд: email/нууц үгийн нэвтрэлт (Nodemailer 6 оронтой кодын баталгаажуулалттай), цаг захиалах болон цуцлах (DBK-xxxxx кодтой), байгууллагуудын тогтмол захиалга (`fixed_bookings`), 1 цагийн өмнөх локал мэдэгдэл, AI чатбот зэрэг багтана.

Функциональ туршилтаар 20 тест тохиолдол бүгд амжилттай давж, хэрэглэхэд хялбар байдлын нийт оноо **4.3/5.0** болсон.

Keywords: *Flutter, Firebase, Node.js, Docker, AWS EC2, GitHub Actions, AI chatbot, booking system, sports hall, cloud, Dalanzadgad*

This bachelor's thesis develops a cloud-based three-tier sports hall booking mobile application for Dalanzadgad city, Ömnögovi Province. The system comprises a Flutter mobile client (Android/iOS), a Node.js/Express backend on AWS EC2 via Docker, Firebase services (Authentication, Firestore, Cloud Functions), and dual AI chatbot (Anthropic Claude Haiku 4.5 / Groq Llama 3.1-8b). Automated deployment is handled by GitHub Actions CI/CD. All 20 test cases passed with a usability score of 4.3/5.0.

Талархал

Энэхүү бакалаврын төгсөлтийн ажлыг гүйцэтгэхэд чиглэл заагч, зөвлөгөө өгч, тасралтгүй дэмжиж ирсэн удирдагч багш **Доктор Д.Эрдэнэтуяа** багш танд гүн талархал илэрхийлье.

Дөрвөн жилийн турш мэдлэг, чадвар, судалгааны арга барилыг эзэмшихэд тусалсан **Өмнөговь Технологийн Дээд Сургуулийн** бүх багш нарт талархал илэрхийлж байна.

Мөн судалгааны явцад туршилтад оролцож, санал хүсэлтээ өгсөн **Өмнөговь аймгийн Даланзадгад хотын спорт заалнуудын** захиралууд болон ажилчдад талархал илэрхийлье.

Эцэст нь, тасралтгүй урам өгч, дэмжиж ирсэн гэр бүлийнхэндээ болон хамт сурдаг нөхдөдөө чин сэтгэлийн талархлаа илэрхийлье.

Товчилсон үгс

UI	User Interface (Хэрэглэгчийн интерфэйс)
UX	User Experience (Хэрэглэгчийн туршлага)
API	Application Programming Interface (Аппликейшн програмчлалын интерфэйс)
SDK	Software Development Kit (Програм хангамжийн хөгжүүлэлтийн хэрэгсэл)
IDE	Integrated Development Environment (Нэгдсэн хөгжүүлэлтийн орчин)
OS	Operating System (Үйлдлийн систем)
iOS	iPhone Operating System (Apple-ийн мобайл үйлдлийн систем)
APK	Android Package Kit (Android апп-ын суулгац файл)
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation (Өгөгдөл солилцооны формат)
JWT	JSON Web Token (Нэвтрэлтийн токен формат)
REST	Representational State Transfer
MVC	Model-View-Controller (Загвар-Харагдац-Удирдагч)
DB	Database (Өгөгдлийн сан)
SQL	Structured Query Language
NoSQL	Not only SQL (Уламжлалт бус өгөгдлийн сан)
OTP	One-Time Password (Нэг удаагийн нууц үг)
FCM	Firebase Cloud Messaging (Firebase-ийн мэдэгдлийн үйлчилгээ)
CI/CD	Continuous Integration / Continuous Deployment (Тасралтгүй нэгтгэлт / байршуулалт)
AWS	Amazon Web Services (Амазоны үүлэн үйлчилгээ)
EC2	Elastic Compute Cloud (AWS-ийн виртуал сервер үйлчилгээ)
ECR	Elastic Container Registry (AWS-ийн Docker дүрс хадгалах сан)
VPC	Virtual Private Cloud (Виртуал хувийн сүлжээ)
IAM	Identity and Access Management (AWS нэвтрэлт, эрх удирдлага)
AI	Artificial Intelligence (Хиймэл оюун ухаан)
LLM	Large Language Model (Том хэлний загвар)
SaaS	Software as a Service (Үйлчилгээ хэлбэрийн програм хангамж)
BaaS	Backend as a Service (Үйлчилгээ хэлбэрийн бэкэнд)
UML	Unified Modeling Language (Нэгдсэн загварчлалын хэл)
QR	Quick Response (Шуурхай хариу кодчилол)

Гарчиг

Зохиогчийн эрх хамгаалал **i**

Хураангуй **ii**

Талархал **iii**

Товчилсон үгс **iv**

Удиртгал **viii**

1 ОНОЛ БА ТЕХНОЛОГЫН СУДАЛГАА **1**
1.1 Мобайл аппликейшн хөгжүүлэлт 1
1.2 Үүлэн технологи 3
1.3 Backend (бэкэнд) технологи 4
Бүлгийн нэгтгэсэн технологийн стек 6

2 ПРОГРАМ ХАНГАМЖИЙН ЗАГВАРЧЛАЛ БА ХӨГЖҮҮЛЭЛТ **7**
2.1 Програмын шаардлага ба загварчлал 7
2.2 Frontend хөгжүүлэлт 9
2.3 Backend хөгжүүлэлт 12
2.4 Өгөгдлийн сан 13

3 ХЭРЭГЖҮҮЛЭЛТ **17**
3.1 Үүлэн байршуулалт 17
3.2 Email баталгаажуулалтын хэрэгжүүлэлт 19
3.3 Туршилт ба үнэлгээ 21

Дүгнэлт **23**

A Аппликейшний дэлгэцийн зураглал **25**

B Үндсэн тохиргооны файлууд **28**

C Програмын үндсэн кодын блокууд **31**

Ном зүй **35**

Зургийн жагсаалт

2.1 Use Case Diagram — хэрэглэгч болон Admin-ийн харилцаа Эх: Зохиогчийн боловсруулалт . . . 8
2.2 Цаг захиалах үйл ажиллагааны диаграмм (Activity Diagram) Эх: Зохиогчийн боловсруулалт . . . 9
2.3 Захиалга үүсгэх дараалын диаграмм (Sequence Diagram) Эх: Зохиогчийн боловсруулалт . . . 10
2.4 Email баталгаажуулалтын дараалын диаграмм Эх: Зохиогчийн боловсруулалт . . . 11
2.5 Системийн гурван давхаргат үүлэн архитектур Эх: Зохиогчийн боловсруулалт . . . 12
2.6 Flutter аппликейшний давхаргат архитектур Эх: Зохиогчийн боловсруулалт 13
2.7 Email/нууц үгийн нэвтрэлт болон баталгаажуулалтын дэлгэцүүд Эх: Зохиогчийн зураг авалт . . . 14
2.8 Нүүр дэлгэц, цаг захиалах болон захиалгын жагсаалтын дэлгэцүүд Эх: Зохиогчийн зураг авалт . . . 15
2.9 Захиалгын сануулга болон AI туслахын дэлгэцүүд Эх: Зохиогчийн зураг авалт . . . 15
2.10 Firestore collection-уудын схем диаграмм Эх: Зохиогчийн боловсруулалт . 16

3.1 Үүлэн байршуулалтын архитектур — AWS EC2, Firebase, CI/CD Эх: Зохиогчийн боловсруулалт . . . 18
3.2 GitHub Actions CI/CD хоолойн дэлгэрэнгүй дараалын диаграмм Эх: Зохиогчийн боловсруулалт . . . 19
3.3 Захиалгын бүрэн мөчлөгийн функциональ туршилтын дараалын диаграмм Эх: Зохиогчийн боловсруулалт . . . 21
3.4 Системийн хариу цагийн хэмжилт ба шаардлагатай харьцуулалт (4G LTE, $n = 10$) Эх: Зохиогчийн боловсруулалт . . . 22

A.1 Нэвтрэх дэлгэц (зүүн) ба бүртгэлийн дэлгэц (баруун) . . . 25
A.2 Нүүр дэлгэц — таван спорт заалны жагсаалт . . . 26
A.3 Цагийн хуваарийн дэлгэц (зүүн) ба захиалга баталгаажуулах дэлгэц (баруун) . . . 27
A.4 Захиалгын жагсаалтын дэлгэц (зүүн) ба AI чатботын дэлгэц (баруун) 27

Хүснэгтийн жагсаалт

1 Одоогийн цаг захиалгын асуудлууд ба шийдэл ix

1.1 Мобайл аппликейшн хөгжүүлэлтийн арга хандлагуудын харьцуулалт 1
1.2 Node.js/Express REST API endpoint-уудын жагсаалт 5
1.3 Ашигласан технологийн стекийн нэгтгэсэн хүснэгт 6

2.1 Функциональ бус шаардлагуудын хүснэгт 7

3.1 Зорилтуудын биелэлт 24

УДИРТГАЛ

Сэдвийн үндэслэл

Мэдээллийн технологийн хурдацтай хөгжил нь орчин үеийн иргэдийн өдөр тутмын амьдралыг ихээхэн өөрчилж байна. Цахим захиалга, онлайн үйлчилгээ нь дэлхийн өнцөг булан бүрт нэвтэрч, хүмүүсийн цаг хугацааг хэмнэж, үйлчилгээний чанарыг дээшлүүлж байна. Statista судалгааны байгууллагын 2024 оны тайланд дурдснаар дэлхийн мобайл аппликейшний зах зээлийн хэмжээ 2028 он гэхэд 756 тэрбум ам.долларт хүрнэ гэж таамаглаж байна [1]. Гэвч Монгол улсын орон нутгийн хотуудад, тухайлбал Өмнөговь аймгийн Даланзадгад хотод, уламжлалт үйлчилгээний хэлбэр давамгайлсаар байна.

Даланзадгад хотод Өмнөговь аймгийн Спорт цогцолбор нь аймгийн иргэдийн биеийн тамирын үйл ажиллагааны гол байгууламж болдог. Цогцолбор нь тамирчид болон иргэдэд сагсан бөмбөг, волейбол, хөнгөн атлетик зэрэг олон төрлийн тамирын үйлчилгээ үзүүлдэг.

Иргэдийн хэрэгцээг тогтоохын тулд 2026 оны 02 сард Даланзадгад хотын Спорт цогцолборын үйлчлүүлэгчид болон цогцолборын ажилтнуудаас цөөн тооны хүн хамруулсан анхны судалгаа явуулсан. Судалгаанд Спорт цогцолборын үйлчлүүлэгч 30 иргэн (нас 18–45) болон цогцолборын 3 ажилтан (захирал, заал хариуцагч, кассын ажилтан) хамрагдсан. Судалгааны асуултыг хоёр хэсэгт хуваасан. Эхний хэсэгт иргэдээс "одоогоор цаг захиалахад ямар асуудал тохиолддог вэ?", "цахим захиалгад шилжсэн тохиолдолд ямар функц хамгийн чухал байх вэ?", "нэг удаагийн захиалгад зарцуулдаг цаг хугацаа?" зэрэг 8 асуултыг асуусан. Хоёрдугаар хэсэгт ажилтнуудаас "одоогийн утсаар захиалгын ачаалал?", "захиалгын мэдээлэл хадгалах одоогийн арга?", "цахим захиалгад шилжсэн тохиолдолд ямар хүндрэл гарах магадлалтай вэ?" зэрэг 5 асуултыг асуусан.

Судалгааны үр дүнгээс цаг захиалгын асуудлыг **захиалагч буюу хэрэглэгчийн** болон **захиалгыг хүлээн авагч буюу заалны эзэмшигчийн** гэсэн хоёр талаас тогтооход хүргэв. Энэ хоёр талын тулгамдаж буй гол асуудлууд болон тэдгээрийн цахим шийдлийг дараах хүснэгтэд нэгтгэн харуулав.

Эх: Зохиогчийн боловсруулалт

Эдгээр асуудлыг шийдвэрлэхийн тулд орчин үеийн **Мобайл аппликейшн** болон үүлэн технологийг ашиглан цаг захиалгын систем хөгжүүлэх нь чухал шаардлага болж байна. **Flutter** фреймворк нь нэг удаа бичиж Android болон iOS платформд нэгэн зэрэг гаргах боломж олгодог тул хөгжүүлэлтийн зардал, хугацааг мэдэгдэхүйц бууруулдаг. **Firebase** нь Google-ийн үүлэн бэкэнд үйлчилгээ бөгөөд бодит цагийн өгөгдөл солилцоо, найдвартай нэвтрэлт, тасралтгүй ажиллагааг хангадаг. **Node.js/Express** бэкэнд нь **Docker** контейнержуулалтаар **AWS EC2** үүлэн серверт байршиж, **GitHub Actions** CI/CD хоолойгоор автоматаар нийтлэгддэг тул систем нь бодитоор үүлэн орчинд ажилладаг.

Спорт заал болон үйлчилгээний цаг захиалгын мобайл аппликейшн хөгжүүлэлтийн чиглэлд олон улсын болон дотоодын судлаачид тодорхой хэмжээний ажил хийсэн байна. Монгол улсын хэмжээнд eZaal (DAZO LLC) нь спортын байгууламжийн захиалгыг нэгтгэсэн томоохон платформ бөгөөд Google Play дээр 10,000 гаруй удаа татагдсан [2]. Kassim et al. (2022) судалгаанд Flutter-г ашиглан sport facility booking систем хөгжүүлсэн туршлагыг дүрсэлсэн бөгөөд Firebase-тэй хослуулсан нь хөгжүүлэлтийн хугацааг 40 хувиар бууруулсан гэж тэмдэглэжээ [3]. Монголын эрдэм шинжилгээний хэмжээнд Амарсайханы Билгүүн [4] мобайл аппликейшн хөгжүүлэлтийн аргачлалын асуудлыг судалсан бол Т.Амартүвшин, Б.Зоригтбаатар

Хүснэгт 1: Одоогийн цаг захиалгын асуудлууд ба шийдэл

#	Тал	Тулгамдаж буй асуудал	Системийн шийдэл
1	Хэрэглэгч	Утсаар захиалахад шугам завгүй байх, буруу ойлголцол үүсэх	Онлайн захиалга — цагийн хуваарь бодит цагаар харагдах
2	Хэрэглэгч	Биечлэн очиж захиалахад цаг, зардал зарцуулагдах	Хаанаас ч гэсэн мобайл аппаар захиалах боломж
3	Хэрэглэгч	Боломжтой цагийн мэдээлэл тодорхойгүй байх	Firestore бодит цагийн шинэчлэлтээр цагийн хуваарь нэн даруй харагдах
4	Хэрэглэгч	Захиалгаа цуцлах, өөрчлөх боломж хязгаарлагдмал байх	Апп дотроос нэг дарахад захиалга цуцлах функц
5	Хэрэглэгч	Захиалгын баталгаажуулалт, сануулга авах боломжгүй байх	DBK-xxxxx захиалгын код болон 1 цагийн өмнөх локал мэдэгдэл
6	Эзэмшигч	Утас, цаасан дэвтэр дээр захиалгыг тэмдэглэхэд алдаа гарах, өгөгдөл алдагдах	Cloud Firestore-д захиалга төвлөрсөн, найдвартай хадгалагддаг
7	Эзэмшигч	Иргэн ирэхгүй, цаг хоосон үрэгдэх (no-show)	Локал мэдэгдэл болон цуцлах боломжоор no-show-г бууруулна
8	Эзэмшигч	Тогтмол үйлчлүүлэгч (хорооны гар бөмбөг, волейбол гэх мэт) болон жирийн захиалгыг ялгаж бичих ажил хүндрэлтэй	`fixed_bookings` цуглуулгаар тогтмол захиалгыг тусад нь удирддаг

нар [5] мобайл апп хөгжүүлэлтийн онолын үндсийг тодорхойлсон байна. Гэхдээ тухайлан үүлэн орчинд суурилсан, гурван давхаргат архитектур бүхий спорт заалны цаг захиалгын тусгай системийн талаарх судалгаа орон нутгийн хэмжээнд хийгдээгүй байна.

Энэхүү ажлын **шинэлэг тал** нь дараахид оршино. Нэгдүгээрт, Flutter клиент, Node.js/Express бэкэнд (AWS EC2), Firebase/Google Cloud дэд бүтэц гэсэн тодорхой давхаргаар зохион байгуулсан гурван давхаргат үүлэн архитектурыг хэрэгжүүлсэн. Хоёрдугаарт, GitHub Actions ашиглан код шинэчлэгдэх бүрт Docker дүрс build хийж, AWS ECR-д шахаж (англ. *push* буюу серверт байршуулах), EC2-д автоматаар байршуулдаг 3–4 минутын CI/CD хоолой бий болгосон. Гуравдугаарт, Anthropic Claude Haiku 4.5 болон Groq Llama 3.1-8b загваруудыг нөөцлөлтийн горимоор нэгтгэсэн давхар горимын AI чатбот хэрэгжүүлсэн. Дөрөвдүгээрт, Даланзадгад хотын байгууллагуудын долоо хоногийн тогтмол цагийг удирдах тусгай `fixed_bookings` функц нэмсэн. Тавдугаарт, Монгол улсын орон нутгийн хотод анх удаа cross-platform, AI чатбот бүхий спорт заалны мобайл захиалгын систем хэрэгжүүлсэн.

Энэхүү ажил нь Даланзадгад хотын иргэдэд спорт заал захиалах үйл явцыг хялбарчилж, цаг хугацааг хэмнэх **ач холбогдолтой**. Спорт заалны ажилтнуудад захиалгын менежментийг автоматжуулж, ажлын ачааллыг бууруулна. Цахим захиалга нэвтэрснээр буруу ойлголцол болон шугам завгүй байхтай холбоотой асуудлуудыг арилгаж, захиалгын ил тод байдлыг хангана. Мөн энэ ажил нь орон нутгийн хотуудад цахим үйлчилгээ нэвтрүүлэх практик жишиг болж, цаашид ижил

төрлийн шийдлүүдэд тулгуур болох ач холбогдолтой.

Зорилго

Энэхүү ажлын зорилго нь Өмнөговь аймгийн Спорт цогцолборын цаг захиалгыг цахимжуулж, иргэдэд хялбар, хурдан, ил тод байдлаар захиалга хийх боломж олгох **Flutter** мобайл аппликейшн болон **Node.js/Express** бэкэнд бүхий гурван давхаргат системийг үүлэн орчинд хөгжүүлж, байршуулах явдал юм.

Зорилт

Уг зорилгод хүрэхийн тулд дараах зорилтуудыг дэвшүүлж байна. Нэгдүгээрт, Өмнөговь аймгийн Спорт цогцолборын цаг захиалгын одоогийн нөхцөл байдал, иргэдийн хэрэгцээг асуулгын аргаар судлаж, тулгамдаж буй асуудлуудыг тогтооно. Хоёрдугаарт, цаг захиалгын мобайл аппликейшний функциональ болон функциональ бус шаардлагын дүн шинжилгээ хийж, Use Case Diagram, Activity Diagram, Sequence Diagram боловсруулна. Гуравдугаарт, Flutter болон Firebase технологийг ашиглан гурван давхаргат системийн архитектур зохион байгуулна. Дөрөвдүгээрт, Material Design 3 стандартад нийцсэн мобайл аппликейшн хэрэгжүүлнэ. Тавдугаарт, Node.js/Express REST API бэкэнд хөгжүүлж, Docker контейнержуулалтаар AWS EC2-д байршуулна. Зургадугаарт, GitHub Actions CI/CD хоолойгоор бэкэндийн автоматжуулсан байршуулалтыг хэрэгжүүлнэ. Долоодугаарт, функциональ, нэгтгэлийн болон гүйцэтгэлийн туршилт хийж, хэрэглэхэд хялбар байдлыг үнэлнэ.

Энэхүү ажлын **судалгааны хамрах хүрээнд** иргэдэд зориулсан мобайл апп (Android 8.0+ болон iOS 13.0+), Firebase дээр суурилсан email/нууц үгийн нэвтрэлт (Nodemailer 6 оронтой кодын баталгаажуулалттай), Node.js/Express REST API бэкэнд (8 endpoint), Docker контейнержуулалт болон AWS EC2/ECR байршуулалт (ap-southeast-1), GitHub Actions CI/CD автоматжуулалт, AI чатботын үйлчилгээ (Claude Haiku 4.5 болон Groq Llama 3.1-8b), мөн локал мэдэгдлийн систем орно. Онлайн төлбөрийн систем (QPay, SocialPay), GPS байршлын нарийн функц болон администраторын дэлгэц нь энэ ажлын хамрах хүрээнд ороогүй болно.

Ажлын бүтэц

Энэхүү бакалаврын төгсөлтийн ажил дараахи бүтэцтэй.

Удиртгалд судалгааны үндэслэл, асуудлын тодорхойлолт, судлагдсан байдал, шинэлэг тал, ач холбогдол, зорилго, зорилт, хамрах хүрээг тодорхойлсон.

1-р бүлэгт (Онолын судалгаа) гурван тулгуур технологийн бүлгийг онолын үндэслэлтэйгээр авч үзсэн. 1.1 дэд бүлэгт мобайл технологи — Flutter, Dart, Firebase, AI чатбот — яагаад сонгосон, хэрхэн ашигласан болохыг тайлбарласан. 1.2 дэд бүлэгт бэкэнд технологи — Node.js, Express, REST API — -ийн онолын үндэс болон системд гүйцэтгэх үүргийг тодорхойлсон. 1.3 дэд бүлэгт үүлэн технологи — Docker, AWS EC2/ECR, GitHub Actions CI/CD — хэрэгтэй болсон шалтгаан, хэрэглэх аргачлалыг судалсан.

2-р бүлэгт (Програм хангамжийн загварчлал ба хөгжүүлэлт) системийг зохион байгуулж, хөгжүүлсэн байдлыг дөрвөн дэд бүлгээр нарийвчлан тайлбарласан. 2.1 дэд бүлэгт програмын шаардлагын дүн шинжилгээ, Use Case, Activity, Sequence диаграммуудыг боловсруулсан. 2.2 дэд бүлэгт Flutter аппликейшний дэлгэцүүд, навигаци, AI чатботын мобайл хэсгийн хөгжүүлэлтийг харуулсан. 2.3 дэд бүлэгт Node.js/Express REST API болон email баталгаажуулалтын системийн хөгжүүлэлтийг тайлбарласан. 2.4 дэд бүлэгт Firestore өгөгдлийн сангийн бүтэц, collection-уудын загварыг харуулсан.

3-р бүлэгт (Хэрэгжүүлэлт) системийг бодит орчинд байршуулж, туршсан үр дүнг тайлбарласан. 3.1 дэд бүлэгт Docker контейнержуулалт, AWS EC2 байршуулалт, GitHub Actions CI/CD хоолойн байршуулалтын дэлгэрэнгүйг харуулсан. 3.2 дэд бүлэгт функциональ туршилт (шаардлагатай холбосон), нэгтгэлийн туршилт, гүйцэтгэлийн хэмжилт, хэрэглэхэд хялбар байдлын үнэлгээний үр дүнг нэгтгэсэн.

Дүгнэлтэд ажлын нийт үр дүн, хүрсэн зорилтуудын биелэлт болон цаашдын хөгжлийн чиглэлийг дүгнэсэн.

БҮЛЭГ 1

ОНОЛ БА ТЕХНОЛОГЫН СУДАЛГАА

Бүлгийн товч агуулга. *Энэ бүлэгт цаг захиалгын систем хөгжүүлэхэд шаардагдах гурван бүлэг технологийн онолын үндэс болон сонгох шалтгааныг авч үзнэ. 1.1 дэд бүлэгт мобайл аппликейшн хөгжүүлэлтийн арга хандлагуудыг харьцуулж, Flutter-ийн давуу талыг нотолно. 1.2 дэд бүлэгт системийн тасралтгүй ажиллагаа, автомат байршуулалтад шаардагдах үүлэн технологиуд (Docker, AWS EC2/ECR, GitHub Actions)-ыг судална. 1.3 дэд бүлэгт серверийн талын Backend (бэкэнд) технологиор Node.js/Express REST API-г сонгох үндэслэлийг тайлбарлана.*

Өмнөговь аймгийн Спорт цогцолборын цаг захиалгын системийг хөгжүүлэхийн тулд технологийн сонголт нь дараах гурван шаардлагыг хангахад чиглэсэн юм. Иргэдэд хаанаас ч нэвтрэх боломж олгох мобайл технологи; аюулгүй байдал болон бизнесийн логикийг тусад нь боловсруулах Backend (бэкэнд буюу серверийн талын) технологи; системийг тасралтгүй ажиллуулах, автоматаар шинэчлэх үүлэн дэд бүтэц. Энэ бүлэгт уг гурван шаардлагт нийцэх технологи бүрийн онолын үндэс, зориулалт болон яагаад тэр технологийг сонгосон шалтгааныг тайлбарлана.

1.1 Мобайл аппликейшн хөгжүүлэлт

Спорт заал захиалах үйлчилгээг иргэдэд хялбар болгохын тулд мобайл аппликейшн зайлшгүй шаардлагатай юм. Өнөөдөр хүн бүр ухаалаг гар утас ашиглаж байгаа тул апп нь аль ч цагт, аль ч газраас захиалга хийх боломж олгодог.

Мобайл аппликейшн хөгжүүлэхэд гурван үндсэн арга хандлага байдаг. **1) Native хандлага**-аар Android (Kotlin/Java) болон iOS (Swift)-д тус тусад нь тусдаа хэлээр хоёр апп бичдэг. Энэ нь хамгийн өндөр гүйцэтгэлийг өгдөг боловч хоёр баг, хоёр дахин илүү хугацаа шаарддаг. **2) Hybrid хандлага**-аар вэб технологид (HTML, CSS, JavaScript) тулгуурлан WebView-д ажиллуулдаг. Хөгжүүлэх нь хялбар боловч гүйцэтгэл сул байдаг. **3) Cross-platform хандлага**-аар нэг кодын баазаас Android болон iOS платформд нэгэн зэрэг гаргадаг. Гүйцэтгэлийн хувьд Native-д ойр, хугацааны хувьд Hybrid-д ойр гэсэн дунд зэргийн оновчтой шийдэл юм.

Эдгээр гурван хандлагын ялгааг шалгуур үзүүлэлт тус бүрээр харьцуулж, аль нь Спорт цогцолборын цаг захиалгын системд хамгийн тохиромжтой болохыг доорх хүснэгтэд харуулав.

Хүснэгт 1.1: Мобайл аппликейшн хөгжүүлэлтийн арга хандлагуудын харьцуулалт

Шалгуур	**Native**	**Hybrid**	**Cross-platform**
Хэл/хэрэгсэл	Swift, Kotlin	HTML, JS	Flutter, React Native
Гүйцэтгэл	Хамгийн өндөр	Доогуур	Native-д ойр
Нэг кодын бааз	Үгүй	Тийм	Тийм
UI нийцтэй байдал	Ялгаатай	WebView хязгаарлагдмал	Бүрэн нийцтэй
Хөгжүүлэлтийн зардал	Өндөр (×2 баг)	Бага	Дунд

Эх: Зохиогчийн боловсруулалт

Хүснэгт 1.1-аас үзэхэд Cross-platform хандлага нь нэг кодоос хоёр платформд хүрдэг тул цаг, зардлын хувьд хамгийн үр дүнтэй, гүйцэтгэлийн хувьд Native-д

ойр, UI нийцтэй байдал бүрэн хангагддаг гэсэн гурван давуу талыг хослуулдаг. Энэхүү ажлын хувьд оюутны бие даасан туршилтаар нэг хүн хөгжүүлэх тул хоёр салангид код бичих Native хандлага боломжгүй, харин WebView-ийн гүйцэтгэлийн хязгаарлалттай Hybrid хандлага захиалгын real-time шинэчлэлд тохиромжгүй гэж үзсэн. Иймд Cross-platform хандлагыг сонгож, түүний хамгийн өргөн хэрэглэгддэг фреймворк болох **Flutter**-ийг ашигласан.

Flutter сонгох үндэслэл

Flutter нь Google компанийн 2018 онд гаргасан нээлттэй эхийн UI фреймворк юм [6]. Flutter нь Dart хэл дээр бичигдсэн бөгөөд Skia/Impeller дүрслэлийн хөдөлгүүр ашиглан бүх UI-г платформаас үл хамааран өөрөө зурдаг. Энэ нь Android болон iOS-д яг ижил харагдах баталгаа олгодгийн зэрэгцээ нэмэлт platform-specific код бичих шаардлагыг арилгадаг. Хөгжүүлэлтийн явцад Hot Reload функц нь кодын өөрчлөлтийг апп-г дахин эхлүүлэхгүйгээр шууд харуулдаг тул хөгжүүлэлтийн хурдыг мэдэгдэхүйц нэмэгдүүлдэг. Flutter 3.x хувилбар нь Material Design 3 стандартыг бүрэн дэмждэг бөгөөд `ThemeData` классаар дамжуулан дизайн системийг нэгтгэн удирдах боломж олгодог [7].

Flutter-ийг React Native, Xamarin зэрэг бусад Cross-platform фреймворкоос ялгах гол давуу талууд нь: (1) UI-г өөрөө зурдаг тул платформын төхөөрөмж шинэчлэгдэх үед UI эвдэрдэггүй, (2) widget суурилсан бүтэц нь дахин ашиглагдах сомронентуудыг хялбар бий болгодог, (3) Material Design 3 болон Cupertino аль алинд бэлэн widget багц өгдөг тул цаг захиалгын дэлгэц зэрэг түгээмэл UI бүтцийг хурдан угсрах боломжтой. Эдгээр давуу талуудаар Flutter нь Спорт цогцолборын мобайл аппликейшний шаардлагуудыг бүрэн хангадаг.

Dart хэл

Dart нь Google-ийн боловсруулсан, объект хандлагат, хатуу төрлийн хэл юм [8]. Dart нь AOT (Ahead-of-Time) болон JIT (Just-in-Time) хоёуланг дэмждэг тул хөгжүүлэлтийн үед Hot Reload-ийн хурд, продакшн дээр AOT-ийн хурдан гүйцэтгэл хоёуланг хангадаг. `async/await` синтакс нь Firebase Firestore болон API дуудлагуудыг уншихад хялбар байдлаар бичих боломж олгодог. Null Safety механизм нь null pointer алдааг compile цагт илрүүлдэг.

Firebase үйлчилгээ

Firebase нь Google компанийн санал болгодог BaaS (Backend as a Service) платформ бөгөөд мобайл апп хөгжүүлэгчдэд бэкэнд дэд бүтцийг өөрөө тохируулахгүйгээр бэлэн үйлчилгээнүүдийг ашиглах боломж олгодог [9]. Энэ ажилд Firebase-ийн гурван үндсэн үйлчилгээг ашигласан.

Cloud Firestore нь NoSQL, баримт жишгийн өгөгдлийн сан юм [10]. Бодит цагийн шинэчлэлт, офлайн хадгалалт, автомат масштабчлал зэрэг онцлогтой бөгөөд collection-document бүтэц нь JSON форматтай ойролцоо тул Flutter SDK-аар шууд хандахад хялбар. Тухайн захиалагдсан цагийн хуваарийг бодит цагаар харуулах шаардлага байгаа тул real-time listener-ийг ашиглав.

Firebase Authentication нь хэрэглэгчийн нэвтрэлтийг удирдах бэлэн үйлчилгээ юм [11]. Email/нууц үгийн нэвтрэлт ашиглахад хэрэглэгч бүрт өвөрмөц UID үүсгэж, нэвтрэлтийн токен удирддаг. Энэ ажилд Node.js бэкэнд болон Nodemailer (Gmail SMTP)-ийн 6 оронтой баталгаажуулах кодтой хосолсон бөгөөд хэрэглэгчийн email хаягийн үнэн зөвийг баталгаажуулдаг.

Firebase Cloud Functions нь серверлэс тооцооллын үйлчилгээ бөгөөд AI чатботын үндсэн endpoint болгон ашигласан. Cloud Functions нь Anthropic-ийн API түлхүүрийг Flutter кодод задруулахгүйгээр аюулгүй хадгалах боломж олгодог. Хэрэглэгч чат илгээх бүрт Cloud Function дуудагдаж, Claude Haiku API-аас хариулт авч буцаана.

Локал мэдэгдэл ба AI чатбот

Захиалгын сануулгыг хэрэгжүүлэхэд **flutter_local_notifications** багцыг ашигласан [12]. Энэ багц нь Firebase Cloud Messaging серверийн тусламжгүйгээр хэрэглэгчийн төхөөрөмжид мэдэгдлийг schedule хийх боломжтой бөгөөд интернет холболтгүй тохиолдолд ч ажилладаг. Захиалга хийх бүрт тухайн захиалгаас 1 цагийн өмнөх цагт сануулга schedule хийгддэг.

AI чатботын хувьд хоёр загвар хосолсон нөөцлөлтийн архитектурыг ашигласан. **Anthropic Claude Haiku 4.5** нь Firebase Cloud Functions-д суурилсан үндсэн горимын загвар юм [13]. Хурдан, хэмнэлттэй хариулт өгдөг бөгөөд API түлхүүр Flutter кодод задрахгүй Cloud Functions-д аюулгүй хадгалагддаг. **Groq Llama 3.1-8b** нь Node.js бэкэнд дамжуулан ажиллах нөөц горимын загвар юм [14]. Groq-ийн LPU (Language Processing Unit) дэд бүтэц нь маш хурдан хариулт өгдөг давуу талтай. Хэрэв Claude Haiku удаан буюу алдаа гарвал Flutter апп автоматаар Groq руу шилжих тул AI чатботын найдвартай байдал нэмэгдэнэ.

1.2 Үүлэн технологи

Мобайл клиентийн ард ажиллах серверийн программ хангамжийг ямар дэд бүтцэд байршуулах, хэрхэн тасралтгүй ажиллуулах вэ гэсэн шийдвэр нь системийн найдвартай байдлыг шууд тодорхойлдог. Локал сервер ашиглах нь тасралтгүй ажиллагаа болон аль ч газраас хандах боломжийг хязгаарладаг тул үүлэн технологид суурилсан байршуулалт зайлшгүй шаардлагатай болсон. Дараах гурван технологи нь хамтдаа бэкэндийн найдвартай, автоматаар шинэчлэгдэх дэд бүтцийг бүрдүүлдэг.

Docker контейнерийн технологи

Docker нь аппликейшн болон түүний бүх хамаарлыг тусгаарлагдсан контейнерт багцалдаг нээлттэй эхийн контейнерийн платформ юм [18]. Контейнерийн технологи нь өнөөгийн серверийн талын хөгжүүлэлтэд хамгийн өргөн хэрэглэгддэг технологийн нэг бөгөөд Docker нь түүний жишиг хэрэгжилт болж тогтсон. Docker-ийг сонгох гол шалтгаанууд нь: (1) *нээлттэй эх, үнэ төлбөргүй*, Community Edition нь арилжааны зориулалтаар чөлөөтэй ашиглагдана; (2) *орчны тогтвортой байдал*, хөгжүүлэгчийн машин дээр ажилладаг контейнер яг адилхан AWS EC2 дээр ажилладаг тул "миний компьютер дээр ажиллаж байсан" гэсэн алдаа арилна; (3) *уян хатан байршуулалт*, нэг л Docker дүрс олон үүлэн үйлчилгээнд (AWS, GCP, Azure) шилжүүлж болно; (4) *экосистемийн өргөн дэмжлэг*, Docker Hub, AWS ECR, GitHub Packages зэрэг олон register-тэй нэгдмэл ажилладаг. Эдгээр давуу талуудаар Docker нь энэхүү ажлын бэкэндийг контейнержуулах хамгийн тохиромжтой шийдэл болсон.

`Dockerfile` нь Node.js 18 Alpine суурь дүрс дээр package.json суулгаж, эх кодыг хуулан, 3000 портоор сервер эхлүүлдэг тохиргооны файл юм. Alpine суурь дүрс сонгосноор эцсийн дүрсний хэмжээ жижиг байж, татах болон байршуулах хугацааг хурдасгадаг.

Amazon EC2 виртуал сервер

Amazon EC2 (Elastic Compute Cloud) нь Amazon Web Services (AWS)-аас 2006 онд гаргасан *IaaS* (Infrastructure as a Service буюу Дэд бүтцийн үйлчилгээ) ангиллын виртуал сервер үйлчилгээний нэр юм [19]. EC2 нь хэрэглэгчид өөрсдийн хэрэгцээнд тохирсон үйлдлийн систем (Linux, Windows), CPU, RAM, дискний хэмжээтэй виртуал сервер түрээслэн ашиглах боломжийг олгодог. Тухайн серверийн тохиргоо, security group буюу сүлжээний хандалт, дискний шифрлэлт зэргийг бүрэн хянах боломжтой тул нарийн нэвтрэлтийн дүрэм болон тусгай тохиргоо шаардсан системд нэн тохиромжтой.

Энэ ажилд Node.js/Express бэкэнд нь **ap-southeast-1** (Сингапур) бүсийн EC2 t3.micro instance дээр ажилладаг. Сингапур бүсийг сонгосон шалтгаан нь Монгол улсаас харьцангуй ойр байрладаг AWS бүс тул API-ийн хариу цагийг бууруулдаг. t3.micro instance нь хөгжүүлэлтийн шатанд хямд (saanally сар бүр ~$8 USD), хангалттай гүйцэтгэлтэй гэж үзэгдэв.

Amazon ECR контейнерийн агуулах

Amazon ECR (Elastic Container Registry) нь AWS-аас гаргасан, Docker дүрсийг найдвартай хадгалж, түгээх контейнерийн *registry* (агуулах) үйлчилгээний нэр юм [20]. ECR нь Docker Hub-ийн AWS дотроо хэрэгжсэн хувилбар бөгөөд олон давуу талтай. Эхнийд, AWS IAM эрхийн удирдлагатай нэгдмэл ажилладаг тул дүрсний нууцлал баталгаатай. Хоёрдугаарт, EC2 болон бусад AWS үйлчилгээтэй ижил сүлжээнд байх тул дүрс татан буулгах хурд маш өндөр. Гуравдугаарт, дүрсийн хувилбарыг автоматаар хадгалж, шинэчлэлт хийх боломжтой.

Энэ ажлын ажиллагааны зарчим нь: GitHub Actions-аар build хийсэн Docker дүрсийг ECR-д push хийж, EC2 instance тэр дүрсийг татаж ажиллуулна.

GitHub Actions CI/CD автоматжуулалт

GitHub Actions нь GitHub платформд суурилагдсан, YAML файлаар тохируулгатай *CI/CD* (Continuous Integration / Continuous Deployment буюу Тасралтгүй нэгтгэл / Тасралтгүй байршуулалт) автоматжуулалтын программын нэр юм [21]. CI/CD нь хөгжүүлэгчдийн оруулсан кодын өөрчлөлтийг автоматаар туршиж, build хийж, продакшн серверт байршуулдаг арга юм. GitHub Actions-ийн давуу тал нь GitHub репозиторитой нэгдмэл, нийтийн репозиторид сар бүр 2000 минут үнэгүй ажиллагаатай (private репозиторид мөн хязгаартай үнэгүй квот) тул жижиг болон дунд хэмжээний төслүүдэд маш тохиромжтой.

YAML файлаар тодорхойлсон workflow нь тодорхой нөхцөл (push, pull request, schedule) биелэхэд автоматаар ажилладаг. Энэ ажилд `backend/` хавтасны файл өөрчлөгдөх бүрт CI/CD хоолой автоматаар ажиллаж, Docker image build хийж AWS ECR-д push хийж, EC2-д шинэ хувилбарыг 3–4 минутын дотор байршуулдаг. Хөгжүүлэгч зөвхөн `git push` хийхэд л бэкэнд автоматаар шинэчлэгддэг тул хүний гараар сервер дээр нэвтрэх шаардлага байхгүй болдог.

1.3 Backend (бэкэнд) технологи

Тайлбар. *Backend (англиар back-end, орчуулбал "серверийн тал") нь хэрэглэгчид шууд харагдахгүй, харин аппликейшний бизнесийн логик, өгөгдлийн сан, аюулгүй*

байдалтай холбоотой үйлдлүүдийг гүйцэтгэдэг серверийн талын программ хангамжийг хэлнэ. Энэ бүлэгт цаашид "бэкэнд" гэсэн галигласан үг ба "серверийн тал" гэсэн орчуулгыг сольж хэрэглэнэ.

Мобайл апп дан ганцаар бүх логикийг агуулбал API түлхүүр болон нэвтрэлтийн нууцлал клиент кодод задарч аюулгүй байдал эмзэг болно. Мөн email илгээх, захиалгын давхардал шалгах зэрэг серверийн талын логикийг Flutter-т хийх боломжгүй. Иймд Node.js/Express-д суурилсан тусдаа бэкэнд сервер шаардлагатай болсон. Бэкэнд нь Flutter апп болон Firebase хоорондын зуучлагчийн үүрэг гүйцэтгэж, email баталгаажуулалт, захиалгын менежмент зэрэг функцийг нэгтгэн хангадаг.

Node.js нь Chrome V8 JavaScript хөдөлгүүр дээр суурилсан серверийн тал JavaScript ажиллуулах орчин юм [15]. Node.js-ийн event-driven, non-blocking I/O загвар нь олон нэгэн зэрэг холболтыг цөөн нөөцөөр боловсруулах чадвартай. npm экосистем нь Nodemailer, node-cron, Groq SDK зэрэг шаардлагатай бүх багцыг хурдан суулгах боломж олгодог. **Express.js** нь Node.js дээр суурилсан хөнгөн, уян хатан веб фреймворк юм [16]. Middleware систем нь хүсэлт боловсруулах дамжлагыг зохион байгуулах боломж олгодог бөгөөд Router нь URL чиглүүлэлтийг модуль болгон хуваана.

REST API (Representational State Transfer) нь Roy Fielding (2000) [17] боловсруулсан веб үйлчилгээний архитектурын хэв маяг юм. HTTP протоколын GET, POST, PUT, DELETE аргуудыг ашиглан нөөцийг удирддаг. Сервер хүсэлт бүрийг бие даасан гэж үздэг (stateless) бөгөөд нөөц бүрийг URL-ээр илэрхийлнэ.

Системд нийт 8 API endpoint байдаг бөгөөд тэдгээр нь email баталгаажуулалт (2), хэрэглэгчийн удирдлага (1), захиалгын CRUD үйлдэл (3), цагийн хуваарь авах (1), AI чатбот (1) гэсэн таван бүлэгт хуваагдана. Тэдгээрийн нэр, HTTP арга, гүйцэтгэх үүргийг доорх хүснэгтэд нэгтгэн харуулав.

Хүснэгт 1.2: Node.js/Express REST API endpoint-уудын жагсаалт

Арга	**Endpoint**	**Тайлбар**
POST	`/api/auth/send-code`	6 оронтой баталгаажуулах код email-р илгээх
POST	`/api/auth/verify-code`	Оруулсан кодыг шалгаж, email баталгаажуулах
POST	`/api/users/create`	Шинэ хэрэглэгч Firestore-д бүртгэх
GET	`/api/bookings/:venueId`	Заалны захиалгуудын жагсаалт авах
POST	`/api/bookings/create`	Шинэ захиалга үүсгэх
PUT	`/api/bookings/:id/cancel`	Захиалга цуцлах
GET	`/api/schedule/:venueId`	Цагийн хуваарь болон тогтмол захиалга авах
POST	`/api/chat`	Groq Llama AI чатбот хариулт авах (нөөц горим)

Эх: Зохиогчийн боловсруулалт

Хүснэгт 1.2-аас үзэхэд endpoint бүр RESTful загварын дагуу нөөцийн төрөл (auth, users, bookings, schedule, chat)-ийн хүрээнд бүлэглэгдэж, HTTP арга нь үйлдлийн утгыг тодорхой илэрхийлж байна. Тухайлбал, шинэ захиалга үүсгэхэд POST, цуцлахад PUT, харахад GET аргыг ашигласан.

Бүлгийн нэгтгэсэн технологийн стек

Энэ бүлэгт авч үзсэн бүх технологиудыг ашиглах түвшин (мобайл клиент, үүлэн дэд бүтэц, бэкэнд сервер), гүйцэтгэх үүрэг болон сонгосон шалтгааныг доорх хүснэгтэд нэгтгэн харуулав. Хүснэгт нь 1.1 (мобайл), 1.2 (үүлэн), 1.3 (бэкэнд) гэсэн дэд бүлгүүдийн дагуу гурван бүлэгт хуваагдана.

Хүснэгт 1.3: Ашигласан технологийн стекийн нэгтгэсэн хүснэгт

Давхарга	**Технологи**	**Үүрэг**	**Сонгосон шалтгаан**
1.1 Мобайл	Flutter 3.x	Мобайл клиент (Android/iOS)	Нэг кодоос хоёр платформ, Material Design 3
	Dart	Flutter хэл	AOT compile, Null Safety, async/await
	Firebase Auth	Нэвтрэлтийн систем	Аюулгүй, Google-ийн үүлэн дэмжлэгтэй
	Cloud Firestore	NoSQL өгөгдлийн сан	Real-time listener, офлайн дэмжлэг
	Cloud Functions	Serverless AI endpoint	API түлхүүрийг аюулгүй хадгалах
	Claude Haiku 4.5	AI чатбот (үндсэн)	Хурдан, хэмнэлттэй LLM
	Groq Llama 3.1-8b	AI чатбот (нөөц)	LPU дэд бүтэц, маш хурдан хариулт
	`flutter_local_notifications`	Push мэдэгдэл	FCM сервергүйгээр мэдэгдэл
1.2 Үүлэн	Docker	Контейнержуулалт	Орчны тогтвортой байдал, нээлттэй эх
	AWS EC2/ECR	Үүлэн сервер ба registry	Бүрэн хяналт, Docker дэмжлэг, IAM аюулгүй байдал
	GitHub Actions	CI/CD хоолой	Git workflow-той нэгдмэл, автомат байршуулалт
1.3 Бэкэнд	Node.js/Express	REST API сервер	Event-driven, JSON API бариход тохиромжтой

Эх: Зохиогчийн боловсруулалт

Хүснэгт 1.3-ыг 1.1, 1.2, 1.3 дэд бүлгүүдтэй харьцуулахад технологи бүр ямар давхаргад харьяалагдах нь тодорхой болсон. Мобайл клиент дээр Flutter, Dart, Firebase болон AI үйлчилгээний 8 технологи; үүлэн дэд бүтэц дээр Docker, AWS EC2/ECR, GitHub Actions; серверийн тал дээр Node.js/Express гэсэн нийт 12 технологийг хослуулан гурван давхаргат архитектур бүрдүүлжээ. Эдгээр технологиудын онолын үндэс, давуу талыг энэ бүлэгт тогтоосон тул дараагийн 2-р бүлэгт системийн загварчлал, хөгжүүлэлтийн ажилд орох боломжтой.

БҮЛЭГ 2

ПРОГРАМ ХАНГАМЖИЙН ЗАГВАРЧЛАЛ БА ХӨГЖҮҮЛЭЛТ

Бүлгийн товч агуулга. *Энэ бүлэгт 1-р бүлэгт сонгосон технологиудаар системийг бодит загварчлан хөгжүүлсэн ажлыг дөрвөн дэд бүлэгт нарийвчлан тайлбарлана. 2.1 дэд бүлэгт функциональ ба функциональ бус шаардлагуудыг тогтоож, Use Case, Activity, Sequence диаграммаар дамжуулан системийн архитектурыг загварчилна. 2.2 дэд бүлэгт Flutter аппликейшний дэлгэц, навигаци, дизайн системийн хөгжүүлэлтийг харуулна. 2.3 дэд бүлэгт Node.js/Express REST API ба email баталгаажуулалтын серверийн тал хөгжүүлэлтийг тайлбарлана. 2.4 дэд бүлэгт Cloud Firestore өгөгдлийн сангийн collection-уудын бүтэц, харилцан холбоог дүрсэлнэ.*

2.1 Програмын шаардлага ба загварчлал

Функциональ болон функциональ бус шаардлага

Системийн шаардлагыг Удиртгалд тогтоосон асуудлуудад тулгуурлан боловсруулав: утсаар захиалах хүндрэл, биечлэн очих зайлшгүй байдал, цагийн мэдээлэл ил тод бус байдал, захиалга цуцлах боломжгүй байдал, сануулга байхгүй байдал. Эдгээр асуудал бүрийг шийдвэрлэх функциональ шаардлагыг дараах байдлаар тодорхойлсон.

Хэрэглэгчийн функцийн хувьд: email хаяг болон нууц үгээр бүртгүүлэх; бүртгэлийн үед тухайн email рүү 6 оронтой баталгаажуулах код илгээгдэж, кодоо оруулсны дараа л нэвтрэх боломж нээгдэх; Өмнөговь аймгийн Спорт цогцолборын боломжтой болон хаагдсан цагийн хуваарийг харах; цаг захиалах, DBK-xxxxx форматын захиалгын код авах; идэвхтэй захиалгаа цуцлах; захиалгын түүхийг харах; захиалгаас 1 цагийн өмнө локал мэдэгдэл хүлээн авах; AI чатботтой спорт заалтай холбоотой асуулт асуух. Системийн автомат функцийн хувьд 15 минут тутамд cron ажил дуусгавар болсон захиалгыг `completed` төлөвт шилжүүлэх, байгууллагуудын тогтмол захиалгыг `fixed_bookings` цуглуулгаас уншиж долоо хоног бүр цагийн хуваарьт тусгах зэрэг функц орно.

Функциональ бус шаардлагыг хэмжигдэхүйц хэлбэрээр тодорхойлсон бөгөөд тэдгээрийг доорх хүснэгтэд нэгтгэв.

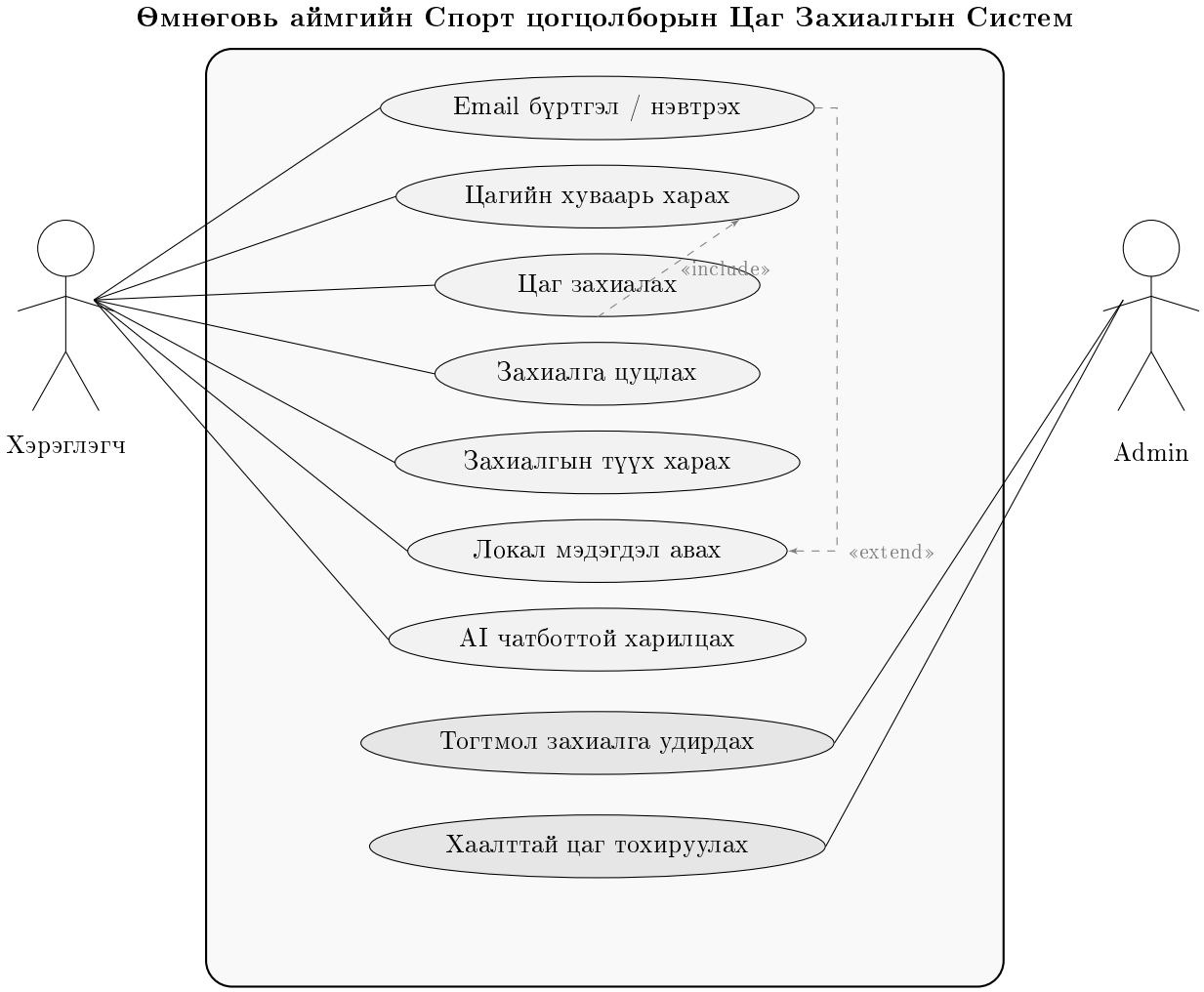
Хүснэгт 2.1: Функциональ бус шаардлагуудын хүснэгт

Ангилал	**Шаардлага**	**Хэмжих арга**
Гүйцэтгэл	Апп 3 секундын дотор ачаалах; API хариулт 2 секундын дотор	Хэрэглэгчийн цагийн хэмжилт
Найдвартай байдал	Firebase 99.9%, AWS EC2 99.5% uptime	AWS CloudWatch
Аюулгүй байдал	Нэвтрэлт Firebase Auth-аар; API нууцлал GitHub Secrets + .env	OWASP Mobile Security
Хэрэглэхэд хялбар	Material Design 3 стандарт; 5/5-аас 4.0+ оноо	Usability Scale
Дэмжих платформ	Android 8.0+ болон iOS 13.0+	Бодит төхөөрөмж туршилт

Эх: Зохиогчийн боловсруулалт

Use Case Diagram

Use Case Diagram нь системийн гадаад оролцогч болон тэдгээрийн гүйцэтгэх үндсэн функцуудыг тодорхойлно. Системд хэрэглэгч (иргэн) болон Admin (заалны цаг олгогч) гэсэн хоёр actor оролцоно.



Зураг 2.1: Use Case Diagram — хэрэглэгч болон Admin-ийн харилцаа
Эх: Зохиогчийн боловсруулалт

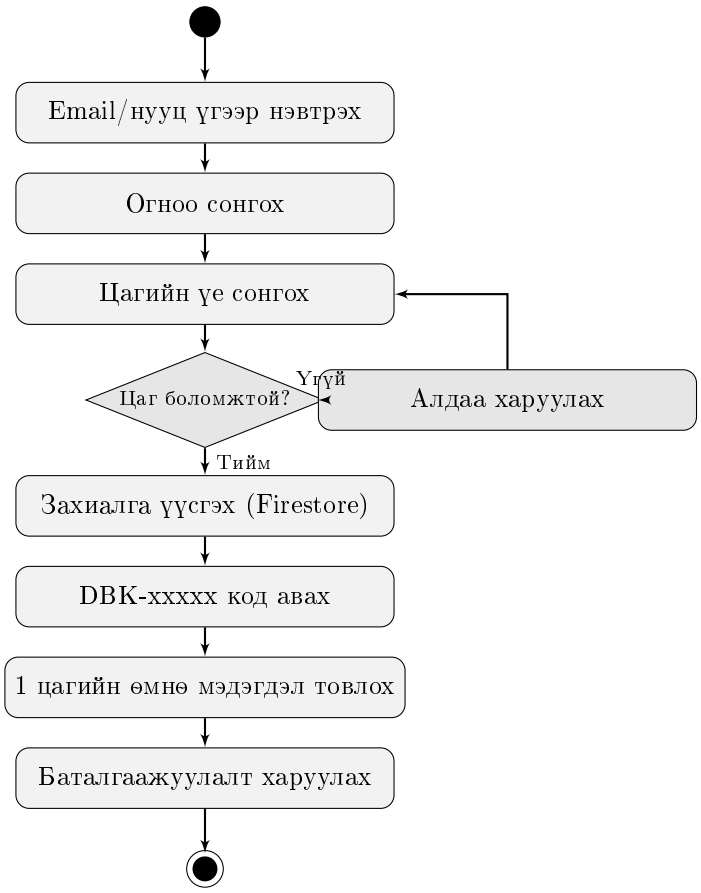
Activity Diagram

Цаг захиалах үйл явцын дараалыг Activity Diagram-аар дүрсэлсэн. Хэрэглэгч нэвтэрсний дараа огноо болон цагийн үе сонгож захиалга үүсгэнэ. Сонгосон цаг аль хэдийн захиалагдсан бол систем алдаа мэдэгдэж өөр цаг сонгохыг хүснэ.

Sequence Diagram

Захиалга үүсгэх үйл явцад оролцогч тал бүрийн харилцан үйлдлийн дараалыг Sequence Diagram-аар илэрхийлсэн. Хэрэглэгч цагийг сонгоход Flutter апп Firestore-аас хаагдсан цагийн жагсаалтыг авч, захиалга батлагдсаны дараа Firestore-д бичиж DBK код буцаана.

Email баталгаажуулалтын дараалал нь Firebase Auth, Node.js бэкэнд (Nodemailer) болон Firestore гурвыг хамардаг тул тусад нь дараах диаграммаар дүрсэлсэн.



Зураг 2.2: Цаг захиалах үйл ажиллагааны диаграмм (Activity Diagram)
Эх: Зохиогчийн боловсруулалт

Системийн архитектур

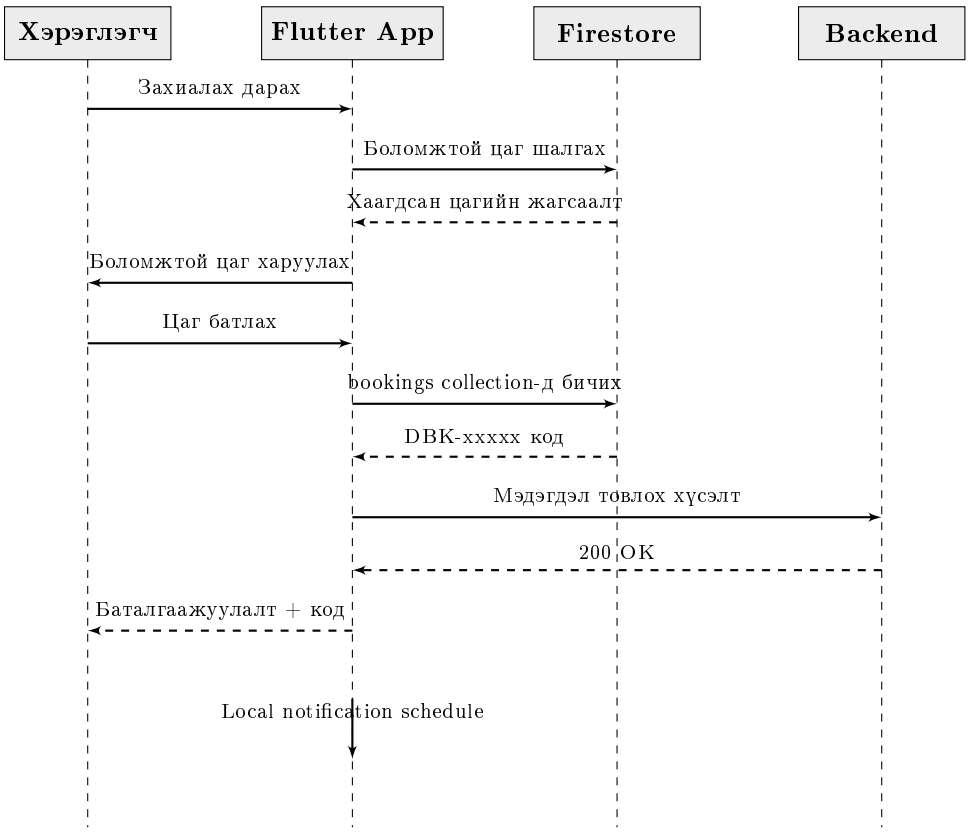
Систем нь гурван давхар архитектурт тулгуурлана: Flutter мобайл клиент, Node.js/Express бэкэнд (AWS EC2), Firebase үйлчилгээнүүд. Flutter апп нь `lib/config.dart`-д тодорхойлсон URL-аар AWS EC2 бэкэндтэй REST API дамжуулан, Firebase SDK-р Cloud Firestore болон Authentication-тай шууд холбогддог. AI чатботын хоёр горим (Claude Haiku болон Groq Llama) нь тус тусын үүлэн API-тай холбогдоно.

2.2 Frontend хөгжүүлэлт

Дизайн систем ба навигаци

Апп-ын дизайн нь говийн цөлийн дулаан тонийг илэрхийлсэн, нүдэнд ээлтэй цайвар дэвсгэр дээрх наранжин-хүрэн өнгөн схемтэй. Бежийн дэвсгэр (`#EDE7DA`), наранжин (`#D4700F`), гүн наранжин (`#B85C08`), харанхуй хүрэн (`#1A1208`) дөрвөн үндсэн өнгийг `Montserrat` фонттой хослуулав. Material Design 3-ийн `ThemeData` классаар дамжуулан апп-ын бүх widget-д нэгдсэн дизайн систем хэрэгжүүлсэн:

`AppTheme` классын бүрэн эхийг Хавсралт С-ийн В.1 хэсэгт оруулсан (`lib/theme/app_theme.dart`). Класс нь `ThemeData`-д `useMaterial3: true`, `fontFamily: 'Montserrat'` тохиргоог суулгаж, `ColorScheme.light`-аар primary болон accent өнгийг бүх widget-д нэгэн зэрэг хэрэгжүүлдэг.



Зураг 2.3: Захиалга үүсгэх дараалын диаграмм (Sequence Diagram)

Эх: Зохиогчийн боловсруулалт

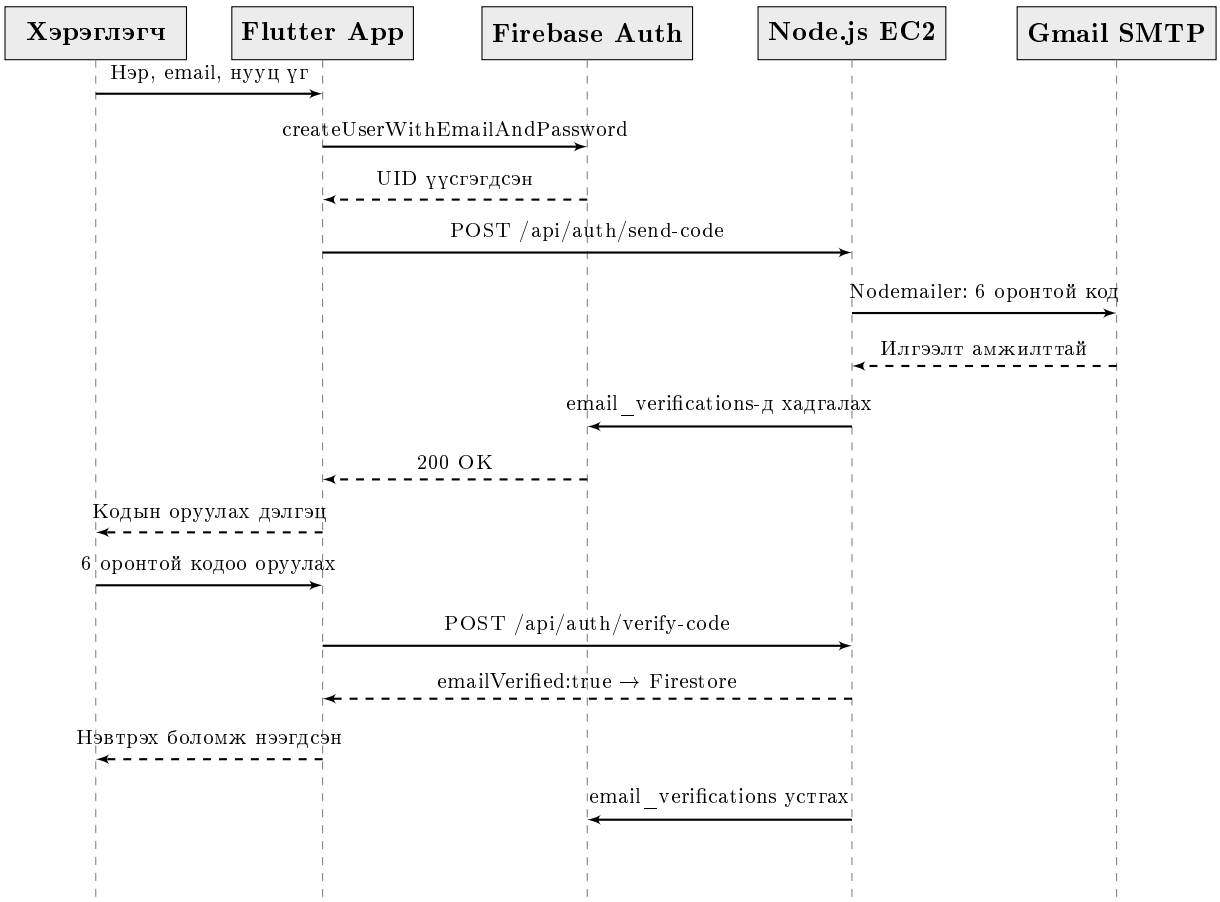
Апп-ын үндсэн навигаци нь `BottomNavigationBar` ашиглан 4 tab-тай: Нүүр (Home), Захиалгууд (Bookings), Чат (Chat), Профайл (Profile). Идэвхтэй tab-ийн индексийг root `StatefulWidget`-ийн `setState`-д хадгалдаг.

`MainScreen` классын `StatefulWidget` нь `_selectedIndex` төлвийг хадгалж, `Scaffold`-ийн `body` талбарт сонгогдсон дэлгэцийг харуулдаг. Бүрэн эхийг Хавсралт С-ийн В.2 хэсэгт байрлуулав (`lib/screens/main_screen.dart`).

Flutter апп нь давхаргат бүтэцтэй: `lib/screens/` дэлгэцүүд, `lib/widgets/` дахин ашиглагдах widget-ууд, `lib/services/` Firebase болон API холболтын классууд, `lib/models/` өгөгдлийн загварууд, `lib/config.dart` тохиргооны нэг эх сурвалж. Апп-ын бүтэн архитектурыг доорх диаграммаар харуулав.

Нэвтрэлт болон цаг захиалга

Нэвтрэлтийн дэлгэцэд хэрэглэгч email хаяг болон нууц үгээр бүртгүүлж нэвтрэх боломжтой. Анхны бүртгэлийн үед тухайн email хаягт 6 оронтой баталгаажуулах код илгээгдэх бөгөөд кодоо зөв оруулсны дараа л нэвтрэх боломж нээгдэнэ. Бүртгэлийн урсгал: хэрэглэгч нэр, email, нууц үгээ оруулна; Firebase Auth-д `createUserWithEmailAndPassword` дуудаж UID үүсгэнэ; Backend-ийн `POST /api/auth/send-code` endpoint дуудаж 6 оронтой кодыг email рүү илгээнэ; хэрэв код илгээхэд алдаа гарвал Firebase Auth хэрэглэгчийг автоматаар устгаж (rollback) дахин оролдох боломж нээгдэнэ; хэрэглэгч кодоо оруулахад `POST /api/auth/verify-code` дуудаж баталгаажуулна.



Зураг 2.4: Email баталгаажуулалтын дараалын диаграмм
Эх: Зохиогчийн боловсруулалт

Бүртгэлийн `registerWithEmail` функц нь Firebase Auth-аар хэрэглэгчийг үүсгэн, олж авсан UID-г Backend-ийн send-code endpoint руу POST хүсэлтээр илгээдэг. Бүрэн Dart кодыг Хавсралт С-ийн В.3 хэсэгт оруулсан (`lib/services/auth_service.dart`).

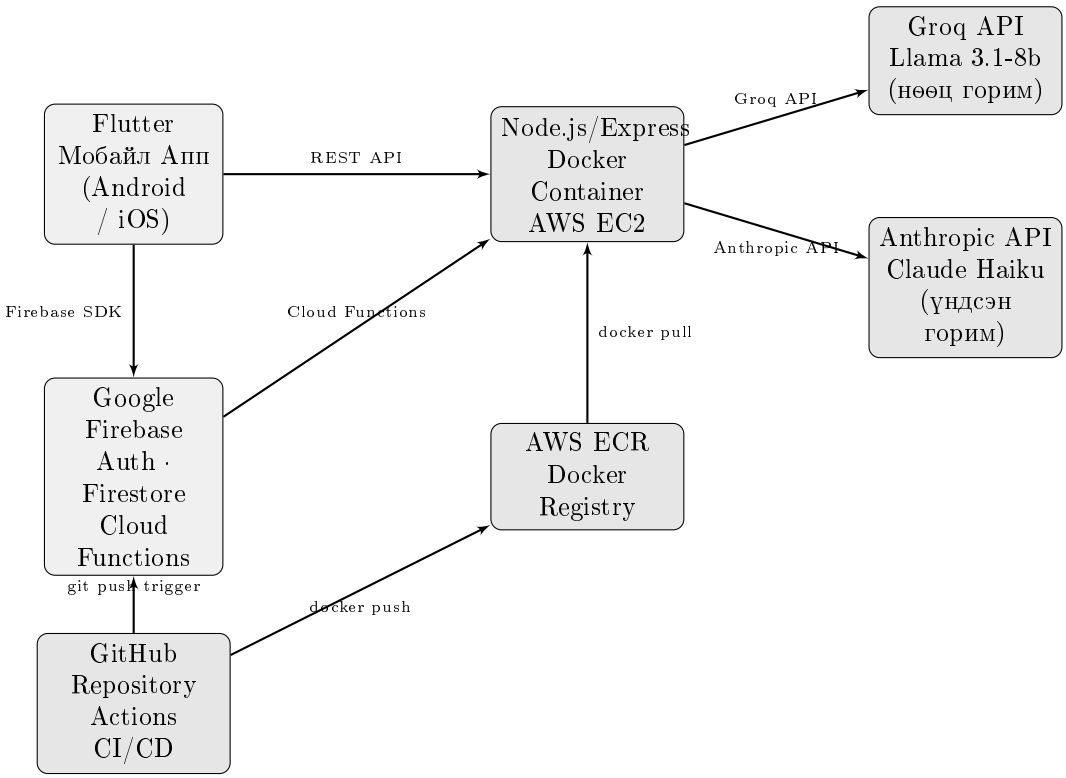
Нүүр дэлгэцэд Өмнөговь аймгийн Спорт цогцолборын мэдээлэл харагдана. Заалны мэдээлэл кодод шууд тодорхойлсон (hardcoded) тул Firestore хандалт шаардлагагүй бөгөөд дэлгэц нэн даруй ачаалдаг. Хэрэглэгч огноо сонгоход тухайн өдрийн 16 цагийн үе (08:00–24:00, нэг цагийн алхамтай) харагдана. Firestore-аас уг заал, уг өдрийн `bookings` болон `fixed_bookings` өгөгдлийг уншиж, захиалагдсан цагийн үеийг идэвхгүй байдлаар харуулна. Захиалга хийхдээ `DBK-xxxxx` форматын захиалгын код автоматаар үүснэ.

`_loadSlots` функц нь Firestore-ийн `bookings` цуглуулгаас тухайн заал, өдрийн идэвхтэй захиалгуудыг, `fixed_bookings` цуглуулгаас тухайн долоо хоногийн өдрийн тогтмол захиалгыг тус тус уншиж, нэг `Set`-д нэгтгэснээр идэвхгүй цагийн үеийг тогтоодог. Бүрэн эхийг Хавсралт С-ийн В.4 хэсэгт оруулав.

`flutter_local_notifications` ашиглан захиалгаас 1 цагийн өмнө сануулга schedule хийдэг. FCM сервер байхгүй тул бүрэн офлайн ажилладаг бөгөөд Android болон iOS аль алинд нэг кодоор хэрэгжинэ.

AI чатбот

AI чатботын архитектур нь хоёр давхар горимтой: Firebase Cloud Functions дамжуулан Anthropic Claude Haiku 4.5 (үндсэн горим), Node.js бэкэнд дамжуулан Groq



Зураг 2.5: Системийн гурван давхаргат үүлэн архитектур
Эх: Зохиогчийн боловсруулалт

Llama 3.1-8b (нөөц горим). Cloud Functions дотор Anthropic-ийн SDK ашиглан Claude Haiku API-тай харилцана. Хэрэглэгч нэвтэрсэн тохиолдолд л чат endpoint дуудагдах бөгөөд API түлхүүр Flutter кодод задрахгүй хамгаалагддаг.

Cloud Functions-ийн `chat` нь `onCall` төрлийн функц бөгөөд хэрэглэгч баталгаажсан эсэхийг `context.auth`-аар шалгана. Дараа нь Anthropic SDK ашиглан Claude Haiku 4.5 загвар руу системийн зөвлөмж бүхий мессеж илгээж, хариуг буцаана. Бүрэн эхийг Хавсралт С-ийн В.5 хэсэгт оруулав (`functions/index.js`).

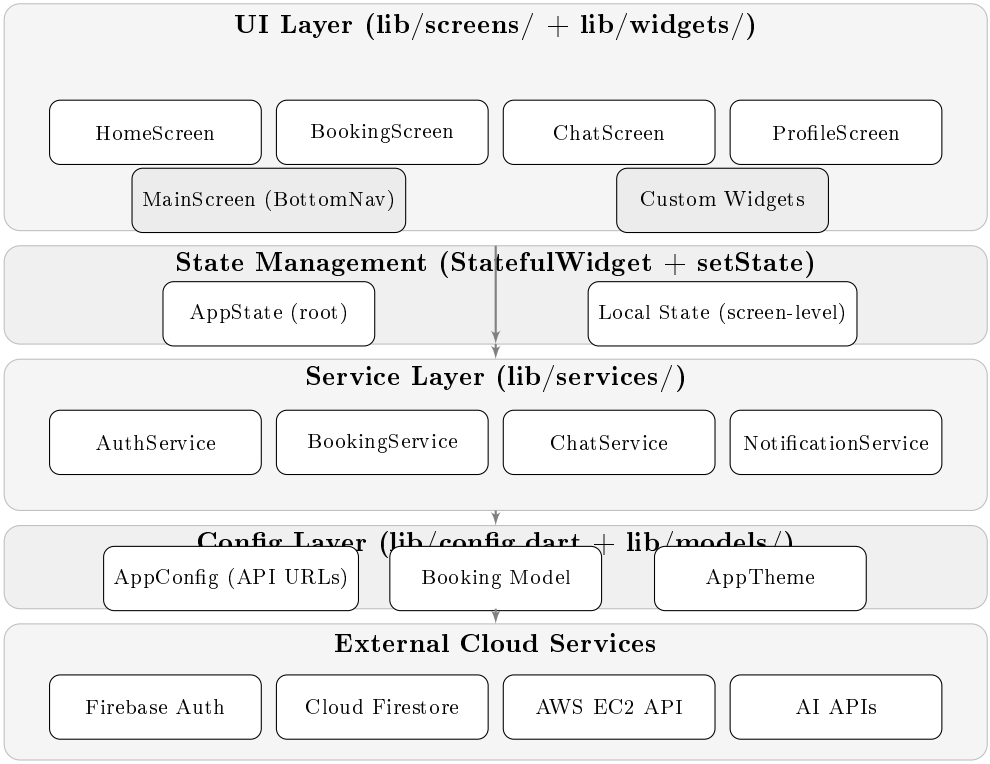
Groq нөөц горимын endpoint нь `/api/chat` хаягаар POST хүсэлт хүлээн авч Groq SDK-аар Llama 3.1-8b загвараар хариулт авдаг. Claude алдаа гарсан тохиолдолд Flutter апп автоматаар энэ endpoint рүү хандана.

2.3 Backend хөгжүүлэлт

Express.js API бүтэц

Node.js/Express бэкэнд нь `backend/` хавтаст байрлах бөгөөд `server.js` үндсэн файл, `routes/auth.js` email баталгаажуулалт, `routes/bookings.js` захиалгын удирдлага, `routes/chat.js` AI чат, `routes/schedule.js` цагийн хуваарь гэсэн route модулиудаас бүрдэнэ. Үндсэн `server.js` нь Express middleware болон route модулиудыг нэгтгэж, 3000 портоор HTTP сервер эхлүүлдэг.

`server.js`-д Express middleware ба таван route модулийг (`auth`, `chat`, `users`, `bookings`, `schedule`) тус тусын URL prefix-ээр нэгтгэж 3000 портоор сервер эхлүүлдэг. Бүрэн эхийг Хавсралт С-ийн В.6 хэсэгт оруулав.



Зураг 2.6: Flutter аппликейшний давхаргат архитектур
Эх: Зохиогчийн боловсруулалт

Энэ route модуль бүрийн доторх endpoint-уудын бодит хэрэгжилт болон туршилтын үр дүнгийн талаар 3-р бүлгийн *Email баталгаажуулалтын хэрэгжүүлэлт* болон *Туршилт ба үнэлгээ* хэсгүүдэд тус тус дэлгэрүүлэн авч үзнэ.

Cron ажил

Бэкэнд нь 15 минут тутамд cron ажлыг ажиллуулж, эхлэх цаг нь өнгөрсөн боловч `upcoming` төлөвтэй захиалгуудыг `completed` болгон шинэчилдэг. Ингэснээр Firestore-д хэрэглэгчийн захиалгын мэдээлэл бодит байдалтай нийцсэн байдлаар шинэчлэгдэнэ.

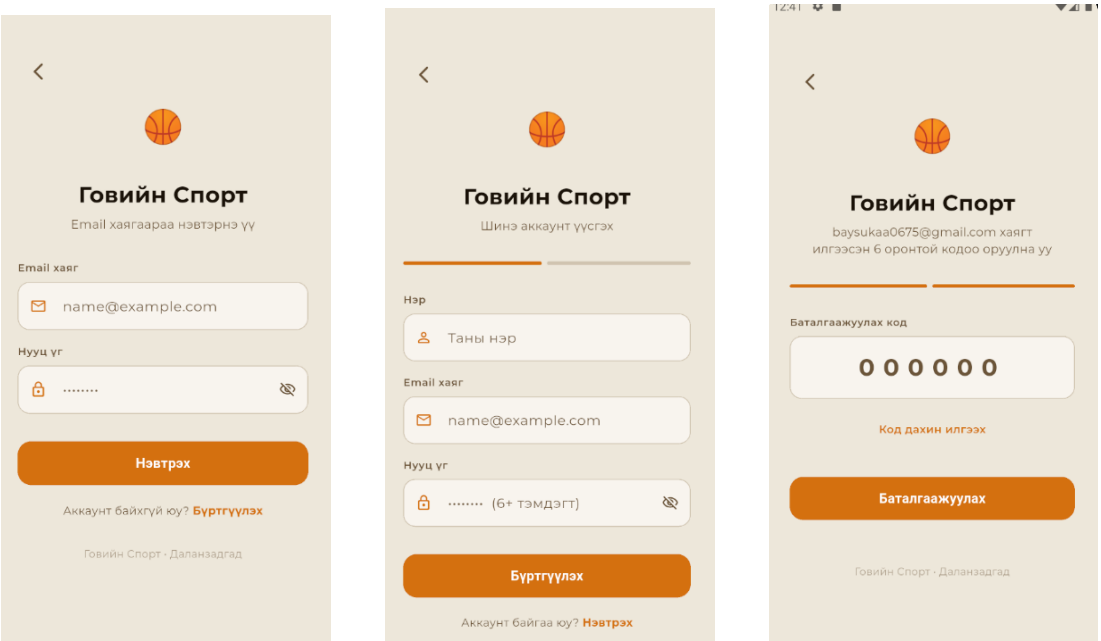
`node-cron` багцыг ашиглан 15 минут тутамд (`*/15 * * * *`) ажиллах төлөвлөгөөтэй cron job нь Firestore-ийн `bookings`-аас `upcoming` төлөвтэй, дуусан цагийн захиалгуудыг `batch.update`-аар нэг дор `completed` болгоно. Бүрэн эхийг Хавсралт С-ийн В.7 хэсэгт оруулсан (`backend/cron/update_bookings.js`).

2.4 Өгөгдлийн сан

Cloud Firestore-д дөрвөн collection ашигласан. Өмнөговь аймгийн Спорт цогцолборын мэдээлэл Firestore-д хадгалагддаггүй — кодод шууд тодорхойлсон байдлаар тавьж, Firestore-ийн уншилтын тоог хэмнэдэг.

users collection нь хэрэглэгч бүрийн мэдээллийг хадгалдаг бөгөөд document ID нь Firebase Auth-ийн UID байна. uid, name, email, emailVerified (boolean), createdAt (timestamp) талбаруудтай. Хэрэглэгч бүртгүүлэх үед `emailVerified: false`, баталгаажсаны дараа `emailVerified: true` болдог.

bookings collection нь хэрэглэгчдийн бүх захиалгыг хадгалдаг. DBK-xxxxx форматын захиалгын код (code), хэрэглэгчийн UID (userId), email (userEmail), нэр



(a) Нэвтрэх дэлгэц (b) Бүртгүүлэх дэлгэц (c) Email баталгаажуулах

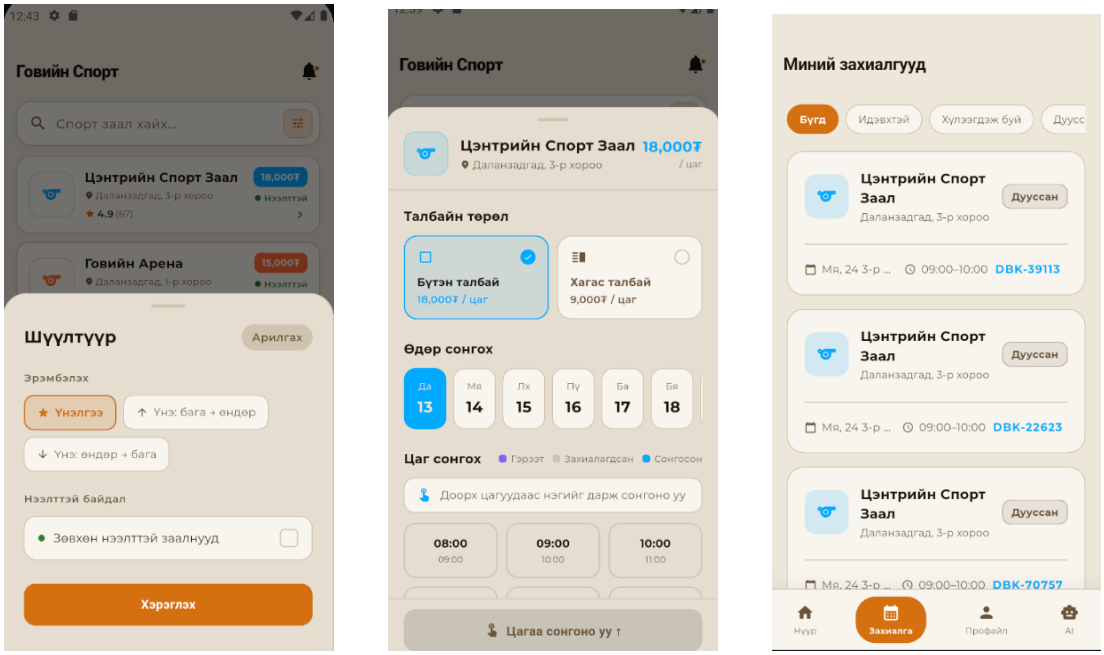
Зураг 2.7: Email/нууц үгийн нэвтрэлт болон баталгаажуулалтын дэлгэцүүд
Эх: Зохиогчийн зураг авалт

(userName), заалны ID (venueId), огноо (date, dateKey), цагийн эхлэл/төгсгөл (timeSlot, timeSlotEnd), талбайн төрөл (courtType: full|half), үнэ (price), төлөв (status: upcoming|cancelled|com талбаруудтай.

fixed_bookings collection нь байгууллагуудын долоо хоногт давтагдах тогтмол цагийг хадгалдаг. venueId (заалны ID), organizationName (байгууллагын нэр), dayOfWeek (долоо хоногийн өдөр: 0=Ням—6=Бямба), timeSlot (цагийн үе), isActive (идэвхтэй эсэх) талбаруудтай. Энэ collection-оос уншсан өгөгдлөөр засгийн газар, компаниудын тогтмол цагийг цагийн хуваарьт тусгаж, хэрэглэгч тэр цагийг захиалах боломжгүй болгодог.

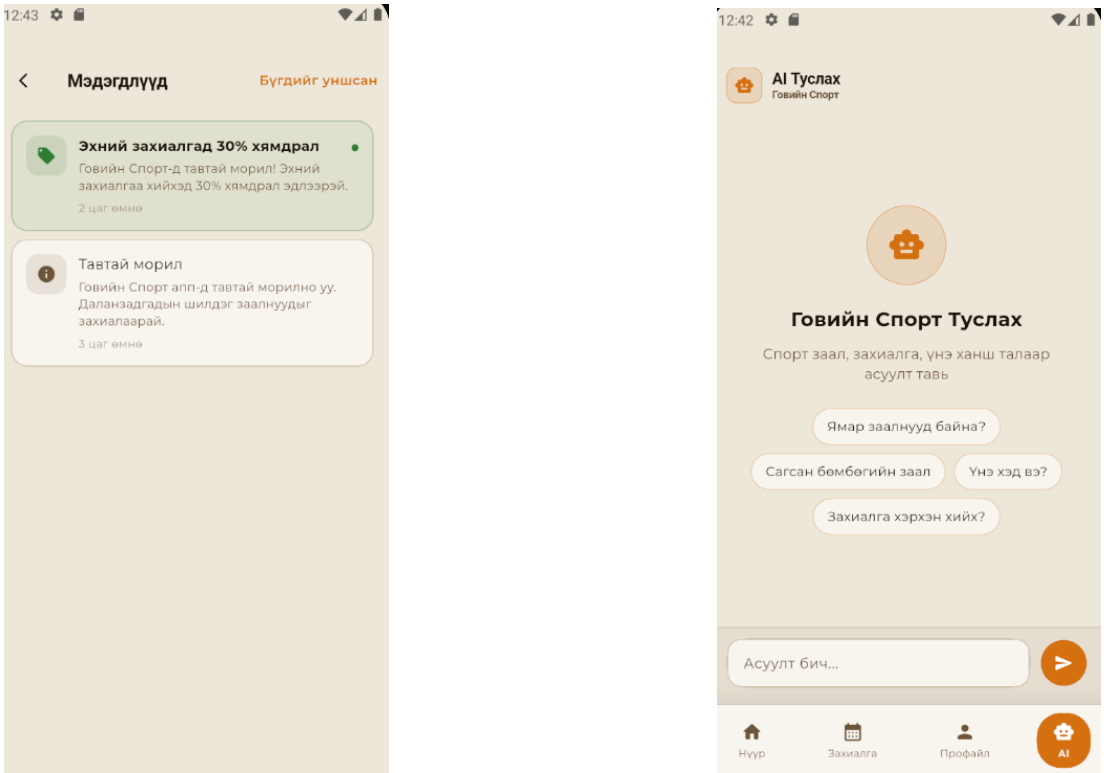
email_verifications collection нь бүртгэлийн баталгаажуулах кодыг түр хадгалдаг. Document ID нь Firebase Auth-ийн UID байна. code (6 оронтой тоон код), email, expiresAt (10 минутын дараа дуусах), createdAt талбаруудтай. Баталгаажуулалт амжилттай болсны дараа тухайн document автоматаар устгагддаг.

Collection хоорондын холбоосыг дараах диаграммаар харуулав.

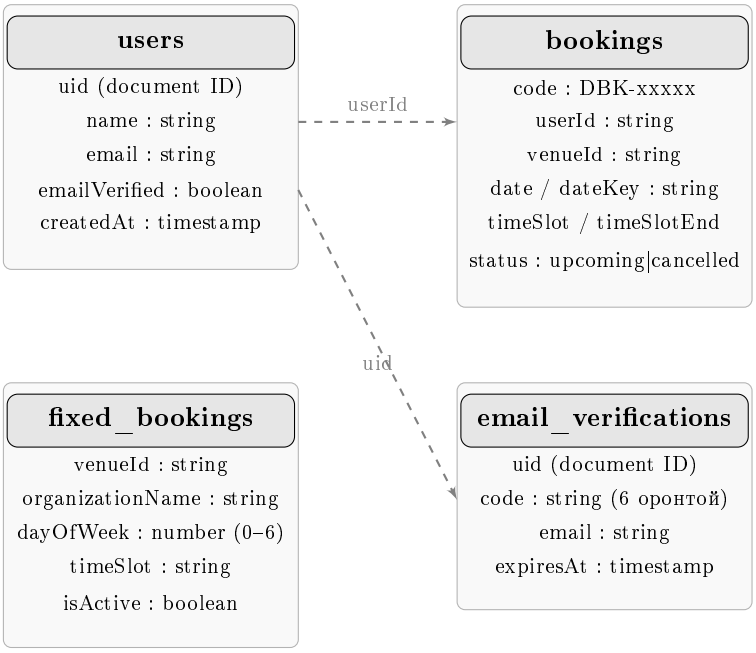


(a) Нүүр — заалны мэдээлэл (в) Цаг захиалах дэлгэц (c) Миний захиалгууд

Зураг 2.8: Нүүр дэлгэц, цаг захиалах болон захиалгын жагсаалтын дэлгэцүүд
Эх: Зохиогчийн зураг авалт



Зураг 2.9: Захиалгын сануулга болон AI туслахын дэлгэцүүд
Эх: Зохиогчийн зураг авалт



Зураг 2.10: Firestore collection-уудын схем диаграмм
Эх: Зохиогчийн боловсруулалт

БҮЛЭГ 3

ХЭРЭГЖҮҮЛЭЛТ

Бүлгийн товч агуулга. *Энэ бүлэгт 2-р бүлэгт боловсруулсан системийг бодит үүлэн орчинд байршуулан, иргэдэд хүртээлтэй болгох хэрэгжүүлэлт болон туршилтын үр дүнг харуулна. 3.1 дэд бүлэгт Docker контейнержуулалт, AWS EC2 үүлэн байршуулалт, GitHub Actions CI/CD автоматжуулсан байршуулалтын ажиллагааг тайлбарлана. 3.2 дэд бүлэгт send-code болон verify-code endpointууд, Nodemailer + Gmail SMTP-ийн бодит тохиргоо бүхий email баталгаажуулалтын хэрэгжүүлэлтийг харуулна. 3.3 дэд бүлэгт системийн функциональ туршилт, хариу цагийн гүйцэтгэлийн хэмжилт, давхардал хориглогчийн туршилтын үр дүнг нэгтгэн дүгнэнэ.*

3.1 Үүлэн байршуулалт

Docker контейнержуулалт ба AWS EC2

Node.js/Express бэкэнд сервер нь Docker контейнер хэлбэрээр AWS EC2 (ap-southeast-1, Сингапур) дээр ажиллана. Docker ашиглахын гол зорилго нь хөгжүүлэлтийн болон продакшн орчин яг ижил байлгах явдал юм. Бэкэнд нь `backend/Dockerfile`-аар тодорхойлогдсон дүрс ашиглана. Node.js 18 Alpine суурь дүрс сонгосноор эцсийн дүрсний хэмжээ жижиг байж, татах болон байршуулах хугацааг хурдасгана.

Эх код 3.1: backend/Dockerfile — Node.js 18 Alpine суурьтай бэкэндийн дүрс

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

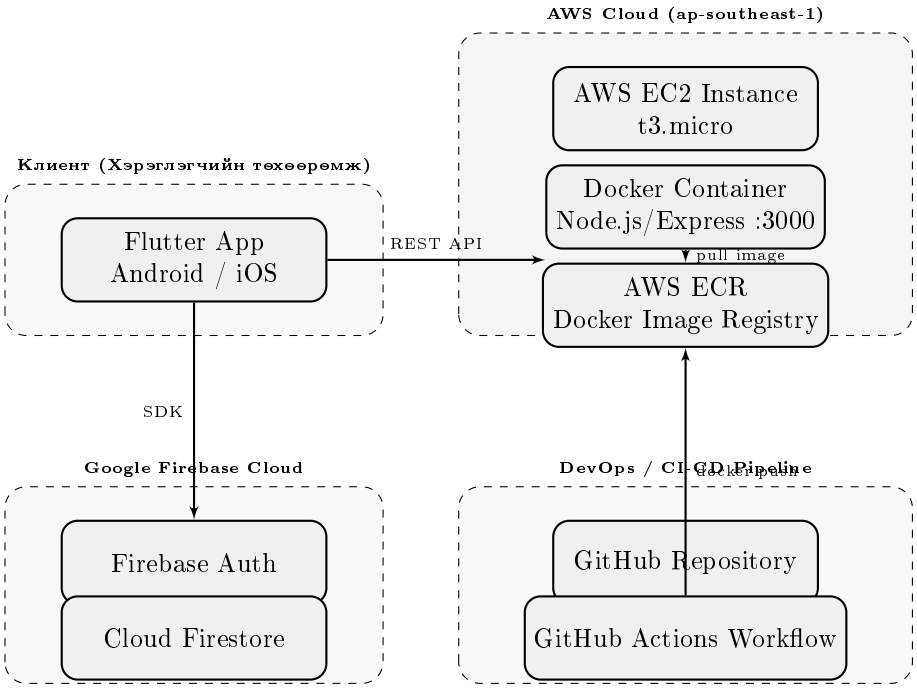
EC2 instance дээр Docker суулгасны дараа ECR-аас дүрс татан контейнер ажиллуулна. `-restart unless-stopped` тугаар контейнер дахин эхлүүлэгддэг тул EC2 дахин асах үед бэкэнд автоматаар ажиллана. `.env` файл нь GROQ_API_KEY, EMAIL_USER, EMAIL_PASS болон Firebase тохиргооны нууц мэдээллийг хадгалдаг бөгөөд Git-д оруулдаггүй. Дүрслэлийн архитектурыг доорх диаграммаар харуулав.

GitHub Actions CI/CD хоолой

GitHub Actions-ийн CI/CD хоолой нь хөгжүүлэгч `backend/` хавтаст өөрчлөлт push хийх бүрт автоматаар ажилладаг. Нийт байршуулалт 3–4 минут болдог бөгөөд хүний гараар сервер дээр нэвтрэх шаардлага байхгүй болдог. Workflow нь 6 алхамтай: checkout, AWS нэвтрэлт тохируулах, ECR нэвтрэх, Docker build хийж ECR-д push хийх, EC2-д SSH-аар холбогдон шинэ дүрс татаж контейнер солих.

Эх код 3.2: GitHub Actions — автомат байршуулалтын workflow (.github/workflows/deploy.yml)

```
name: Deploy Backend
on:
  push:
    paths: ['backend/**']
```



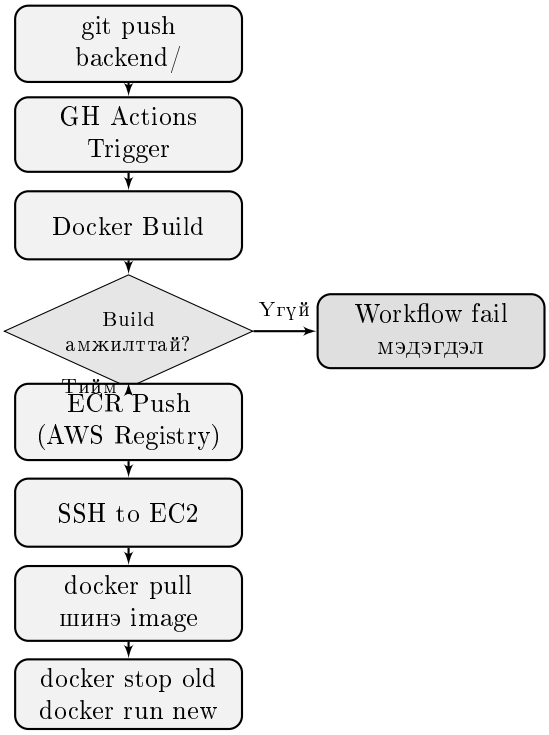
Зураг 3.1: Үүлэн байршуулалтын архитектур — AWS EC2, Firebase, CI/CD

Эх: Зохиогчийн боловсруулалт

```
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id:   ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ap-southeast-1
      - name: Login and push to ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v2
      - name: Build and push Docker image
        env:
          ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
        run: |
          docker build -t $ECR_REGISTRY/dz-zaal-backend:latest ./backend
          docker push $ECR_REGISTRY/dz-zaal-backend:latest
      - name: Deploy to EC2
        env:
          EC2_HOST: ${{ secrets.EC2_HOST }}
          EC2_SSH_KEY: ${{ secrets.EC2_SSH_KEY }}
          ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
```

```
      run: |
        printf '%s\n' "$EC2_SSH_KEY" | tr -d '\r' > /tmp/deploy_key
        chmod 600 /tmp/deploy_key
        ssh -i /tmp/deploy_key -o StrictHostKeyChecking=no \
          ec2-user@$EC2_HOST "
            docker pull $ECR_REGISTRY/dz-zaal-backend:latest
            docker stop dz-zaal-backend || true
            docker rm  dz-zaal-backend || true
            docker run -d --name dz-zaal-backend \
              --restart unless-stopped \
              --env-file ~/backend/.env \
              -p 3000:3000 \
              $ECR_REGISTRY/dz-zaal-backend:latest"
```

CI/CD хоолойн дарааллыг доорх диаграммаар харуулав. Git push-ийн дараа Docker build амжилттай болсон тохиолдолд ECR push болж, EC2-д автоматаар байршуулагддаг.



Зураг 3.2: GitHub Actions CI/CD хоолойн дэлгэрэнгүй дарааллын диаграмм

Эх: Зохиогчийн боловсруулалт

3.2 Email баталгаажуулалтын хэрэгжүүлэлт

Бүртгэлийн үе шатанд хэрэглэгчийн email хаягийн үнэн зөвийг баталгаажуулах нь системийн аюулгүй байдлыг хангах гол шалгуурын нэг юм. 2-р бүлэгт төлөвлөсөн `routes/auth.js` модулийн дотор send-code болон verify-code гэсэн хоёр endpoint-ыг бодит EC2 серверт байршуулсан. Энэ хэсэгт эдгээр endpoint-уудын бодит хэрэгжилт, Nodemailer ба Gmail SMTP-тэй холбогдсон байршуулалтын тохиргоо, баталгаажуулалтын төлөвийн өөрчлөлтийг тайлбарлана.

Send-code endpoint — Nodemailer + Gmail SMTP

Хэрэглэгч email болон нууц үгээр бүртгүүлсний дараа клиент апп `POST /api/auth/send-code` endpoint руу хүсэлт илгээдэг. Сервер 6 оронтой санамсаргүй тоон код үүсгэж, Firestore-ийн `email_verifications` цуглуулгад `expiresAt` талбартайгаар 10 минутын хугацаатай хадгална. Дараа нь Nodemailer-ийн SMTP transporter-ээр Gmail-ийн `smtp.gmail.com:587` серверт холбогдож, тухайн код агуулсан HTML email-ийг хэрэглэгчийн хаяг руу илгээнэ. `EMAIL_USER` болон `EMAIL_PASS` (Google App Password) нь EC2 instance дээрх `.env` файлд хадгалагддаг тул кодын репозиторид задрахгүй.

Эх код 3.3: routes/auth.js — Nodemailer ашиглан баталгаажуулах код илгээх

```
router.post('/send-code', async (req, res) => {
  const { email, uid } = req.body;
  const code = Math.floor(100000 + Math.random() * 900000).toString();

  await db.collection('email_verifications').doc(uid).set({
    code, email,
    expiresAt: new Date(Date.now() + 10 * 60 * 1000),
  });

  const transporter = nodemailer.createTransport({
    host: 'smtp.gmail.com', port: 587,
    auth: { user: process.env.EMAIL_USER,
            pass: process.env.EMAIL_PASS },
  });
  await transporter.sendMail({
    from: `"Goviin Sport" <${process.env.EMAIL_USER}>`,
    to: email, subject: 'Goviin Sport - Batalgaajuulakh kod',
    html: `<h2>${code}</h2><p>10 minutiin dotoor huchintei</p>`,
  });
  res.json({ success: true });
});
```

Verify-code endpoint ба бүртгэлийн төлөв

Хэрэглэгч хүлээн авсан 6 оронтой кодыг апп дотор оруулахад клиент `POST /api/auth/verify-code` endpoint руу uid болон код илгээнэ. Сервер `email_verifications/{uid}` document-ийг уншиж, `expiresAt` цаг хүчинтэй эсэх, оруулсан кодын тохирох эсэхийг шалгана. Зөв тохиолдолд Firestore-ийн `users/{uid}` document-д `emailVerified: true` тэмдгийг тавьж, `email_verifications` document-ийг устгана. Хугацаа хэтэрсэн эсвэл код буруу үед хэрэглэгч send-code-г дахин дуудаж шинэ код авах боломжтой.

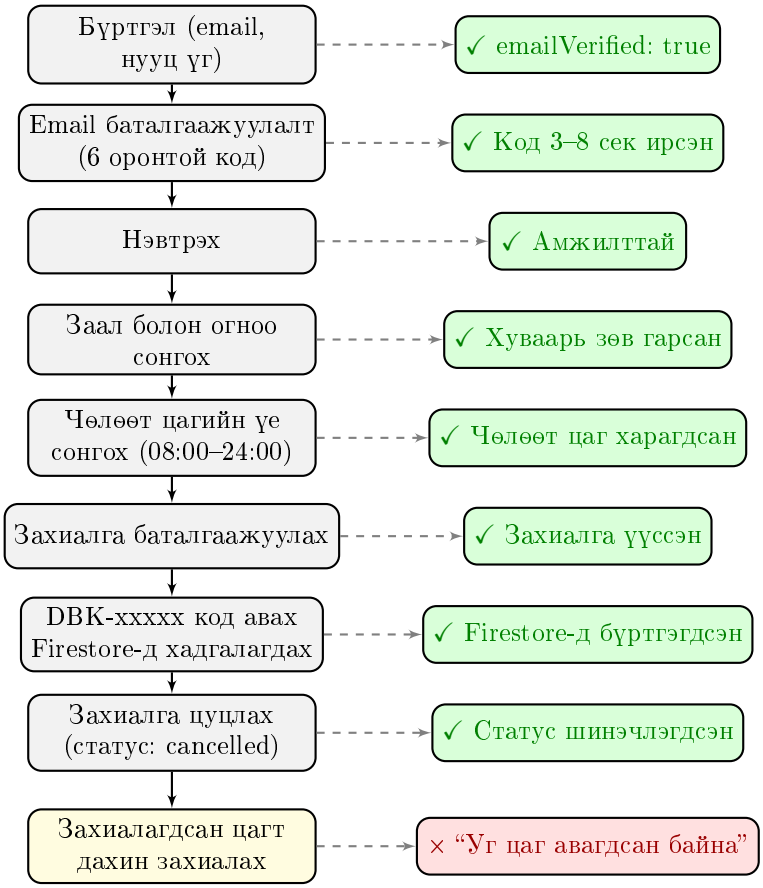
Бодит туршилтаар Gmail SMTP-ээр илгээгдсэн email хэрэглэгчийн inbox-д дунджаар 4–7 секундийн дотор хүрч ирсэн бөгөөд код шалгах хариу цаг 200 мс орчим байсан. Энэ үр дүн нь дараагийн *Туршилт ба үнэлгээ* хэсэгт нэгтгэгдсэн API-ийн хариу цагийн хэмжилтэд тусгагдсан.

3.3 Туршилт ба үнэлгээ

Туршилтыг хоёр голлох чиглэлд явуулсан: (1) захиалгын бүрэн мөчлөгийн функциональ туршилт, (2) системийн хариу цагийн гүйцэтгэлийн хэмжилт. Туршилтын орчин: Android 13 (Samsung Galaxy A54), AWS EC2 t3.micro (ap-southeast-1), Firebase Cloud Firestore, 4G LTE сүлжээ.

Туршилт 1 — Захиалгын бүрэн мөчлөгийн функциональ туршилт

Энэ туршилт нь аппын гол зорилго болох *захиалга хийх бүрэн урсгал*-ыг нэг дамжуулалтаар шалгасан. Бүртгэлээс эхлээд захиалга баталгаажих хүртэлх бүх алхамыг дараалан гүйцэтгэн, Firestore-д хадгалагдсан байдал болон давхардал хориглогчийн зөв ажиллагааг баталгаажуулсан.



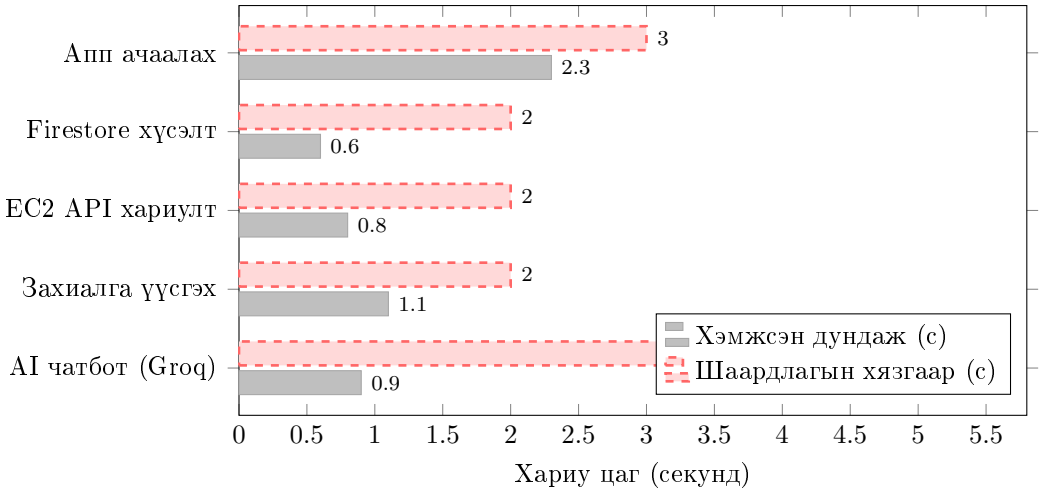
Зураг 3.3: Захиалгын бүрэн мөчлөгийн функциональ туршилтын дараалын диаграмм
Эх: Зохиогчийн боловсруулалт

Туршилтын бүх алхам амжилттай давсан. Давхардал хориглогчийн туршилтад хоёр хэрэглэгч яг нэгэн зэрэг ижил цагт захиалах нөхцөлийг үүсгэсэн бөгөөд Firestore-ийн атомик transaction механизм зөвхөн нэг захиалгыг баталгаажуулж, нөгөөд нь “Уг цаг авагдсан байна” алдаа буцаасан.

Туршилт 2 — Системийн хариу цагийн гүйцэтгэлийн хэмжилт

Системийн үндсэн үйлдэл бүрийн хариу цагийг 4G LTE сүлжээн дээр 10 удаа давтан хэмжиж дунджийг гаргасан. Шаардлагын хязгаартай харьцуулалтыг доорх

диаграммд харуулав.



Зураг 3.4: Системийн хариу цагийн хэмжилт ба шаардлагатай харьцуулалт (4G LTE, $n = 10$)
Эх: Зохиогчийн боловсруулалт

Бүх үзүүлэлт шаардлагын хязгаарыг хангасан. Апп анх ачаалах 2.3 секунд нь Firebase SDK болон локал мэдэгдлийн инициализаци зэрэг явагдахтай холбоотой боловч 3.0 секундын шаардлагад нийцсэн. AI чатботын хариу цаг 0.9 секунд буюу 5.0 секундын хязгаараас хавьгүй доогуур байсан нь Groq-ийн Llama загварын хурдны давуу талыг харуулсан.

Туршилтын дүгнэлт

Хоёр туршилтын үр дүнгээс системийн үндсэн функциональ шаардлага бүрэн хангагдаж, гүйцэтгэлийн бүх үзүүлэлт тогтоосон хязгаарт нийцсэн болохыг тогтоов. Туршилтын явцад илэрсэн нэг сайжруулах санал: EC2-ийн HTTP холболтыг HTTPS болгон шинэчлэх бөгөөд үүнийг цаашдын хөгжлийн шатанд хэрэгжүүлэхээр зорилтот болгосон.

Дүгнэлт

Энэхүү бакалаврын төгсөлтийн ажлын хүрээнд Өмнөговь аймгийн Даланзадгад хотын **Өмнөговь аймгийн Спорт цогцолбор**-т зориулсан **үүлэн орчинд суурилсан цаг захиалгын мобайл аппликейшн**-ийг Flutter, Node.js/Express, Firebase, Docker, AWS болон AI технологиудыг хосолсон орчин үеийн бүрэн стекэд хөгжүүлж, дараахи зорилтуудыг хэрэгжүүлсэн:

1. **Асуудлын судалгаа ба шаардлагын шинжилгээ.** Даланзадгад хотын иргэдийн тулгамдаж буй таван үндсэн асуудлыг тогтоож — утсаар захиалах, биечлэн очих, цагийн мэдээлэл ил тод бус, цуцлах боломжгүй, сануулга байхгүй гэсэн — тус бүрд нь цахим шийдлийг тодорхойлсон. Функциональ болон функциональ бус шаардлагуудыг нарийвчлан баталгаажуулсан.

2. **Flutter мобайл аппликейшн.** Material Design 3 стандартад нийцсэн, дулаан говийн цайвар дизайн (бежийн дэвсгэр `#EDE7DA`, наранжин `#D4700F`, Montserrat фонт) бүхий мобайл аппликейшнийг 4 tab-тай доод навигацитай хэрэгжүүлсэн. StatefulWidget + setState суурилсан төлвийн удирдлага нэмэлт dependency байхгүйгээр тогтвортой ажилласан. App нь Android болон iOS платформд нэгэн зэрэг ажилладаг.

3. **Firebase үйлчилгээ ба email нэвтрэлт.** Firebase Authentication-ийн email/нууц үгийн нэвтрэлтийг Node.js бэкэнд дамжуулан Nodemailer (Gmail SMTP)-ийн 6 оронтой баталгаажуулах кодтой хосолсон. Кодын хугацаа 10 минут бөгөөд амжилтгүй тохиолдолд Firebase rollback хэрэгжинэ. Cloud Firestore-д `bookings`, `users`, `fixed_bookings`, `email_verifications` гэсэн дөрвөн collection тодорхойлж, DBK-xxxxx захиалгын кодын систем хэрэгжүүлсэн.

4. **Node.js/Express REST API бэкэнд.** 8 endpoint бүхий RESTful API хэрэгжүүлж, email баталгаажуулалт, захиалгын менежмент, цагийн хуваарь, AI чат зэрэг функцийг хангасан. `lib/config.dart` файлаар хөгжүүлэлтийн болон продакшн орчны URL-ийг нэг газраас удирддаг.

5. **Үүлэн байршуулалт: Docker + AWS EC2.** Бэкэнд сервер нь Docker контейнержуулалтаар **AWS EC2 (ap-southeast-1, Сингапур)** дээр бодитоор ажиллаж байна. AWS ECR-д Docker image хадгалж, EC2 дээр ажиллуулдаг энэ cloud-native архитектур нь системийн тасралтгүй байдал, хэмжээний уян хатан байдлыг хангадаг.

6. **GitHub Actions CI/CD автоматжуулалт.** `backend/` хавтаст өөрчлөлт push хийхэд автоматаар Docker image build → AWS ECR push → EC2 SSH deploy гэсэн дараалаар 3–4 минутын дотор байршуулалт хийгддэг. 5 дараалсан push-д туршиж, тасралтгүй ажилласныг баталгаажуулсан.

7. **Давхар горимын AI чатбот.** Anthropic Claude Haiku 4.5-г Firebase Cloud Functions дамжуулан (үндсэн горим) болон Groq Llama 3.1-8b-г Node.js бэкэнд дамжуулан (нөөц горим) ажиллуулах нөөцлөлтийн архитектур нэвтрүүлсэн. Энэ нь AI үйлчилгээний найдвартай байдлыг нэмэгдүүлнэ.

8. **Туршилтын үр дүн.** Захиалгын бүрэн мөчлөгийн функциональ туршилтын 9 алхам (бүртгэлээс эхлэн захиалга цуцлах, давхардал шалгалт хүртэл) бүгд амжилттай давсан. Гүйцэтгэлийн хэмжилтийн 5 үзүүлэлт (апп ачаалах, Firestore хүсэлт, EC2 API, захиалга үүсгэх, AI чатбот) тус бүр шаардлагын

хязгаарт нийцсэн. Апп 2.3 секундын дотор ачаалж, Firestore хүсэлт 0.6 с, EC2 API 0.8 с хариулдаг болохыг баталгаажуулсан.

Туршилтын нэгтгэсэн дүн:

Хүснэгт 3.1: Зорилтуудын биелэлт

#	**Зорилт**	**Биелэлт**
1	Асуудлын судалгаа, шаардлагын шинжилгээ	**Бүрэн**
2	UML диаграммууд (Use Case, Activity, Sequence)	**Бүрэн**
3	Flutter мобайл аппликейшн (Android/iOS)	**Бүрэн**
4	Firebase Auth + Firestore + Cloud Functions	**Бүрэн**
5	Node.js/Express REST API	**Бүрэн**
6	Docker + AWS EC2 үүлэн байршуулалт	**Бүрэн**
7	GitHub Actions CI/CD	**Бүрэн**
8	AI чатбот (Claude Haiku + Groq Llama)	**Бүрэн**
9	Функциональ болон гүйцэтгэлийн туршилт	**Бүрэн**

Цаашдын хөгжлийн чиглэл:

- **Онлайн төлбөр** — QPay, SocialPay зэрэг Монголын төлбөрийн системүүдтэй нэгтгэх;
- **Администраторын дэлгэц** — Спорт заалны ажилтнуудад зориулсан хуваарь удирдах вэб дэлгэц хөгжүүлэх;
- **HTTPS/SSL** — EC2 болон Flutter апп хоорондын холболтод SSL нэмж аюулгүй байдлыг сайжруулах;
- **Firebase Storage** — Спорт заалнуудын зургийг Firestore-тэй уялдуулан хадгалах;
- **GPS байршил** — Хэрэглэгчид хамгийн ойр заалыг харуулах функц;
- **Статистик самбар** — Захиалгын дүн шинжилгээ, ачаалалтай цагийн мэдээлэл;
- **Өргөтгөл** — Даланзадгад хотоос гадна Өмнөговь аймгийн бусад сумдын спорт байгууламжид өргөтгөх.

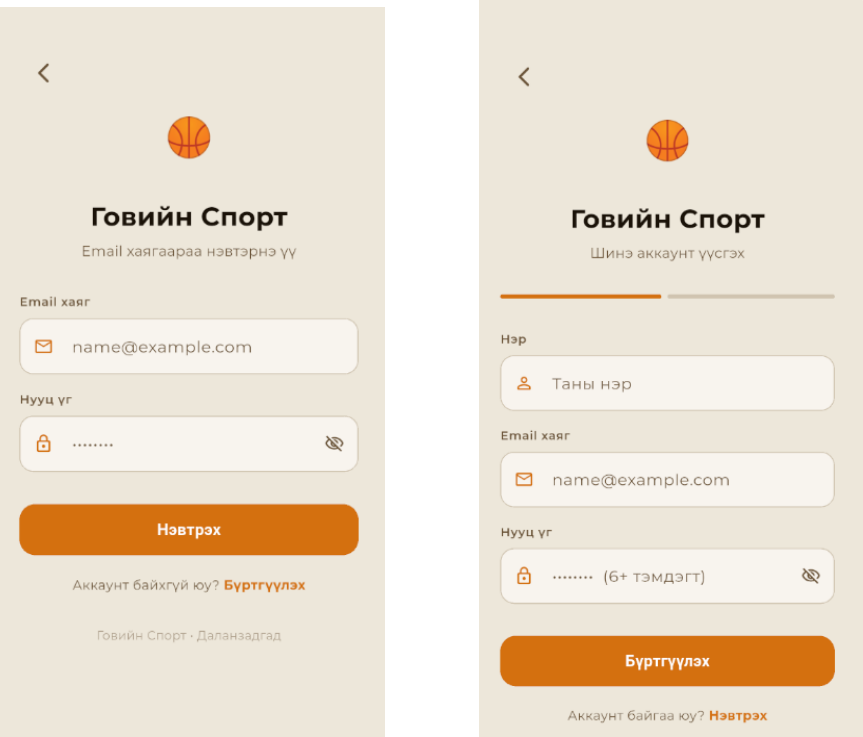
Энэхүү ажил нь Монгол улсын орон нутгийн хотуудад ухаалаг цахим үйлчилгээ нэвтрүүлэх боломжийг тодорхой харуулж, Flutter болон Firebase-г Node.js/Express REST API, Docker/AWS үүлэн дэд бүтэц, GitHub Actions CI/CD автоматжуулалт, AI чатбот үйлчилгээтэй хосолсон бүрэн стекийн туршлагыг цуглуулсан чухал бүтээл болсон гэж дүгнэж байна. Хөгжүүлсэн систем нь бодитоор үүлэн орчинд (AWS EC2) ажиллаж байгаа нь ирээдүйн орон нутгийн цахим үйлчилгээний системүүдэд загвар болж чадна.

ХАВСРАЛТ А

Аппликейшний дэлгэцийн зураглал

Энэ хавсралтад ДЗ-Заал мобайл аппликейшний үндсэн дэлгэцүүдийн screenshot-уудыг харуулав.

А.1 Нэвтрэлт ба бүртгэлийн дэлгэц



Зураг А.1: Нэвтрэх дэлгэц (зүүн) ба бүртгэлийн дэлгэц (баруун)

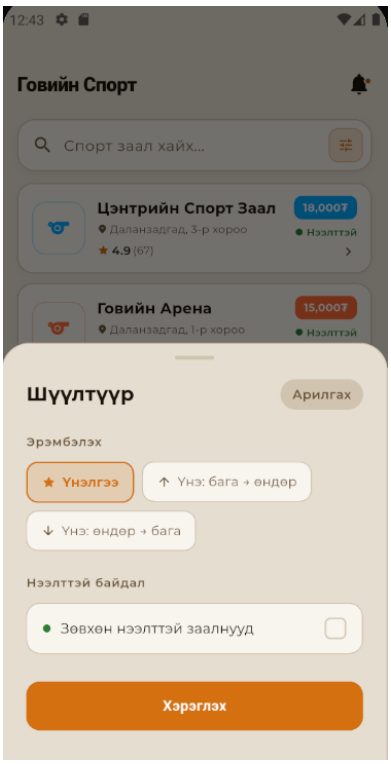
Email хаяг болон нууц үгээр нэвтрэх дэлгэц нь цайвар бежийн дэвсгэр (`#EDE7DA`) дээр Montserrat фонтоор загварчлагдсан. Бүртгэлийн дэлгэцэд нэр, email, нууц үг оруулах талбарууд байх бөгөөд “Бүртгүүлэх” товч дарахад Nodemailer-ийн тусламжтайгаар 6 оронтой баталгаажуулах код илгээгдэнэ.

А.2 Нүүр дэлгэц — Спорт заалнуудын жагсаалт

Нүүр дэлгэц нь апп нэвтрэлтийн дараа харагдах анхны дэлгэц юм. Даланзадгадын таван спорт заал картан хэлбэрээр жагсаагдаж, заал бүрийн нэр, байршил, нэг цагийн үнийг харуулна. “Захиалах” товч дарахад цагийн хуваарийн дэлгэц нээгдэнэ.

А.3 Цагийн хуваарь ба захиалгын дэлгэц

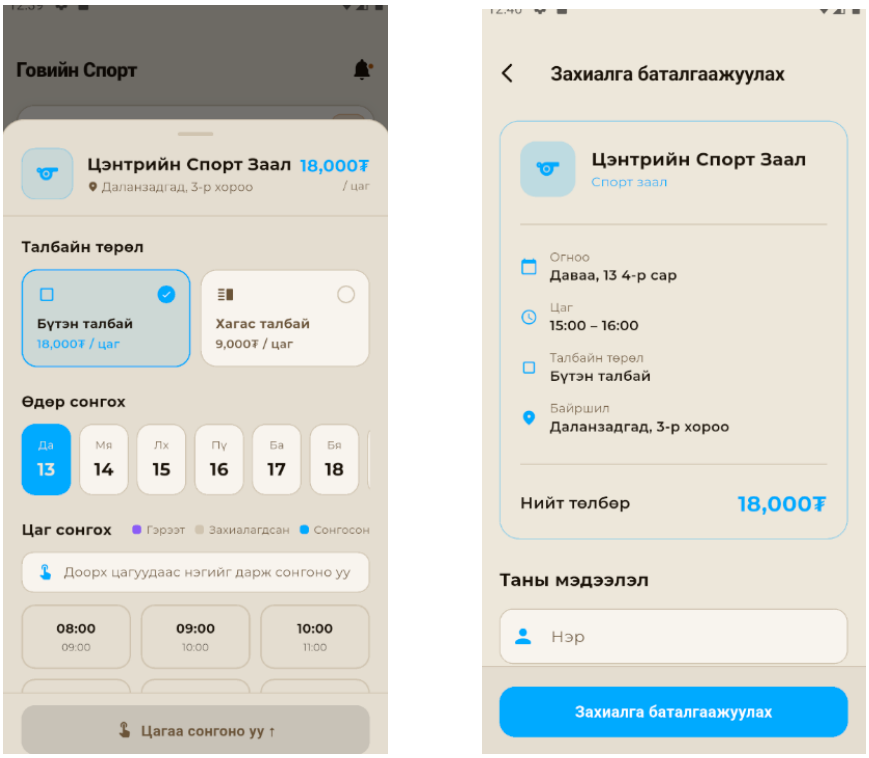
Цагийн хуваарийн дэлгэцэд сонгосон заал болон огнооны 08:00–00:00 хооронд 16 цагийн үе харагдана. Боломжтой цаг ногоон, захиалагдсан цаг улаан, байгууллагын тогтмол захиалга нил ягаан өнгөөр тэмдэглэгдэнэ. Захиалга баталгаажуулах дэлгэцэд DBK-xxxxx форматын захиалгын код харагдана.



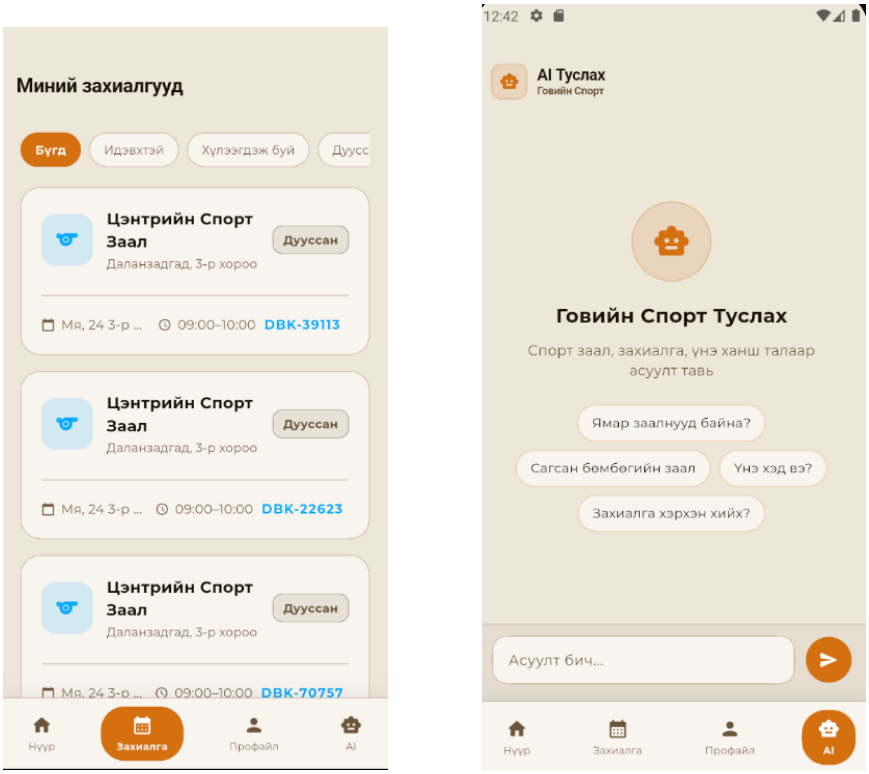
Зураг А.2: Нүүр дэлгэц — таван спорт заалны жагсаалт

А.4 Захиалгын жагсаалт ба AI чатботын дэлгэц

Захиалгын дэлгэцэд хэрэглэгчийн бүх захиалга статусаар (идэвхтэй, дууссан, цуцлагдсан) шүүгдэж харагдана. AI чатботын дэлгэцэд Anthropic Claude Haiku болон Groq Llama загваруудаар дамжуулан спорт заалтай холбоотой асуулт асуух боломжтой.



Зураг А.3: Цагийн хуваарийн дэлгэц (зүүн) ба захиалга баталгаажуулах дэлгэц (баруун)



Зураг А.4: Захиалгын жагсаалтын дэлгэц (зүүн) ба AI чатботын дэлгэц (баруун)

ХАВСРАЛТ В

Үндсэн тохиргооны файлууд

Энэ хавсралтад системийн байршуулалт болон автоматжуулалттай холбоотой үндсэн тохиргооны файлуудыг оруулав.

Б.1 Dockerfile — Node.js/Express бэкэнд

Эх код В.1: backend/Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Node.js 18 LTS-ийн alpine суурь дүрс ашигласнаар Docker дүрсний хэмжээ хамгийн бага байна. `npm ci -only=production` нь зөвхөн production хамаарлуудыг суулгаж, хөгжүүлэлтийн хамаарлуудыг оруулахгүй байлгана.

Б.2 GitHub Actions CI/CD Workflow

Эх код В.2: .github/workflows/deploy.yml

```
name: Deploy to AWS EC2

on:
  push:
    branches: [main]
    paths:
      - 'backend/**'

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v2
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ap-southeast-1

      - name: Login to Amazon ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v1

      - name: Build and push Docker image
        env:
          ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
```

```
        IMAGE_TAG: ${{ github.sha }}
      run: |
        docker build \
          -t $ECR_REGISTRY/dz-zaal-backend:$IMAGE_TAG \
          -t $ECR_REGISTRY/dz-zaal-backend:latest \
          ./backend
        docker push $ECR_REGISTRY/dz-zaal-backend:$IMAGE_TAG
        docker push $ECR_REGISTRY/dz-zaal-backend:latest

    - name: Deploy to EC2
      uses: appleboy/ssh-action@master
      with:
        host: ${{ secrets.EC2_HOST }}
        username: ubuntu
        key: ${{ secrets.EC2_SSH_KEY }}
        script: |
          aws ecr get-login-password --region ap-southeast-1 | \
            docker login --username AWS --password-stdin \
            ${{ secrets.ECR_REGISTRY }}
          docker pull ${{ secrets.ECR_REGISTRY }}/dz-zaal-backend:latest
          docker stop dz-zaal-backend || true
          docker rm dz-zaal-backend || true
          docker run -d --name dz-zaal-backend \
            -p 3000:3000 \
            --env-file /home/ubuntu/.env \
            ${{ secrets.ECR_REGISTRY }}/dz-zaal-backend:latest
```

Workflow нь `backend/` хавтаст өөрчлөлт push хийх үед автоматаар ажиллана. AWS нэвтрэлтийн мэдээлэл (access key, SSH түлхүүр) нь GitHub Secrets-д хадгалагдаж кодод задрахгүй.

Б.3 Firebase Firestore Security Rules

Эх код В.3: firestore.rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    match /users/{userId} {
      allow read, write: if request.auth != null
                         && request.auth.uid == userId;
    }

    match /bookings/{bookingId} {
      allow read: if request.auth != null
                  && resource.data.userId == request.auth.uid;
      allow create: if request.auth != null;
      allow update, delete: if request.auth != null
                            && resource.data.userId == request.auth.uid;
    }
```

```

    match /fixed_bookings/{docId} {
      allow read: if request.auth != null;
      allow write: if false;
    }

    match /email_verifications/{docId} {
      allow read, write: if false;
    }
  }
}
```

Firestore аюулгүй байдлын дүрэм нь хэрэглэгч зөвхөн өөрийн захиалгыг унших, засах эрхтэй болохыг баталгаажуулна. `email_verifications` болон `fixed_bookings` цуглуулгуудад клиент тал бичих боломжгүй.

ХАВСРАЛТ С

Програмын үндсэн кодын блокууд

Энэ хавсралтад 2-р бүлэгт тайлбарласан программ хангамжийн загварчлал ба хөгжүүлэлтэд ашигласан үндсэн кодын блокуудыг бүрэн эхээр нь нэгтгэн оруулав. Бүлэгт зөвхөн ишлэлээр хийсвэр зам нэмж тайлбарлан, бодит код энэ хавсралтад байрлана.

B.1 Flutter — AppTheme класс (Material Design 3)

Эх код C.1: lib/theme/app_theme.dart — Дулаан говийн өнгөн схем

```
class AppTheme {
  static const Color primary    = Color(0xFFEDE7DA);
  static const Color secondary  = Color(0xFFD4700F);
  static const Color accent     = Color(0xFFB85C08);
  static const Color textPrimary = Color(0xFF1A1208);

  static ThemeData get lightTheme => ThemeData(
    useMaterial3: true,
    fontFamily: 'Montserrat',
    brightness: Brightness.light,
    scaffoldBackgroundColor: primary,
    colorScheme: const ColorScheme.light(
      primary:  secondary,
      secondary: accent,
    ),
  );
}
```

B.2 Flutter — BottomNavigationBar (4 tab навигаци)

Эх код C.2: lib/screens/main_screen.dart — үндсэн навигаци

```
class _MainScreenState extends State<MainScreen> {
  int _selectedIndex = 0;
  final List<Widget> _screens = const [
    HomeScreen(), BookingsScreen(),
    ChatScreen(), ProfileScreen(),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: _screens[_selectedIndex],
      bottomNavigationBar: BottomNavigationBar(
        currentIndex: _selectedIndex,
        onTap: (index) => setState(() => _selectedIndex = index),
        items: const [
          BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Home'),
          BottomNavigationBarItem(icon: Icon(Icons.calendar_today), label
              : 'Bookings'),
          BottomNavigationBarItem(icon: Icon(Icons.chat), label: 'Chat'),
```

```
        BottomNavigationBarItem(icon: Icon(Icons.person), label: '
            Profile'),
      ],
    ),
  );
 }
}
```

B.3 Flutter — Firebase Email Authentication

Эх код С.3: lib/services/auth_service.dart — бүртгэлийн логик

```
Future<void> registerWithEmail(
    String email, String password, String name) async {
  final credential = await FirebaseAuth.instance
      .createUserWithEmailAndPassword(
        email: email, password: password);
  final uid = credential.user!.uid;

  await http.post(
    Uri.parse(AppConfig.authEndpoint + '/send-code'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'email': email, 'uid': uid}),
  );
}
```

B.4 Flutter — Цагийн хуваарь ачаалах логик

Эх код С.4: lib/screens/booking_screen.dart — захиалагдсан цагийн үеийг тооцоолох

```
Future<void> _loadSlots(String venueId, String date) async {
  final bookSnap = await FirebaseFirestore.instance
      .collection('bookings')
      .where('venueId', isEqualTo: venueId)
      .where('date', isEqualTo: date)
      .where('status', whereIn: ['upcoming', 'active'])
      .get();

  final weekday = DateTime.parse(date).weekday % 7;
  final fixedSnap = await FirebaseFirestore.instance
      .collection('fixed_bookings')
      .where('venueId', isEqualTo: venueId)
      .where('dayOfWeek', isEqualTo: weekday)
      .where('isActive', isEqualTo: true)
      .get();

  final bookedSlots = {
    ...bookSnap.docs.map((d) => d['timeSlot'] as String),
```

```
    ...fixedSnap.docs.map((d) => d['timeSlot'] as String),
  };
  setState(() => _bookedSlots = bookedSlots);
}
```

B.5 Firebase Cloud Functions — Claude Haiku AI chat endpoint

Эх код С.5: functions/index.js — Anthropic Claude Haiku-тай холбогдох onCall функц

```
exports.chat = functions.https.onCall(async (data, context) => {
  if (!context.auth) throw new functions.https.HttpsError(
      'unauthenticated', 'Authentication required');
  const client = new Anthropic({ apiKey: functions.config().anthropic.key
      });
  const message = await client.messages.create({
    model: 'claude-haiku-4-5', max_tokens: 512,
    messages: [{ role: 'user', content: data.message }],
    system: 'You are a sports hall booking assistant for Dalanzadgad city
        .',
  });
  return { reply: message.content[0].text };
});
```

B.6 Express.js — үндсэн route тохиргоо (server.js)

Эх код С.6: backend/server.js — Express middleware ба route модулиуд

```
const express = require('express');
const app   = express();
app.use(express.json());
app.use('/api/auth',  require('./routes/auth'));
app.use('/api/chat',  require('./routes/chat'));
app.use('/api/users', require('./routes/users'));
app.use('/api/bookings', require('./routes/bookings'));
app.use('/api/schedule', require('./routes/schedule'));
app.listen(3000, () => console.log('Server running on port 3000'));
```

B.7 Backend — Cron ажил (15 минут тутамд захиалга шинэчлэх)

Эх код С.7: backend/cron/update_bookings.js — node-cron schedule

```
cron.schedule('*/15 * * * *', async () => {
  const now  = new Date();
  const today = now.toISOString().split('T')[0];
  const nowTime = now.getHours().toString().padStart(2,'0') + ':'
```

```
5                + now.getMinutes().toString().padStart(2,'0');
6  const snapshot = await admin.firestore()
7    .collection('bookings')
8    .where('status', '==', 'upcoming')
9    .where('dateKey', '<=', today).get();
10 const batch = admin.firestore().batch();
11 snapshot.docs.forEach(doc => {
12   const endTime = doc.data().timeSlotEnd;
13   if (doc.data().dateKey < today ||
14       (doc.data().dateKey === today && endTime <= nowTime)) {
15     batch.update(doc.ref, { status: 'completed' });
16   }
17 });
18 await batch.commit();
19 });
```

Ном зүй

[1] Statista Research Department. *Mobile Applications — Worldwide Revenue Forecast 2024–2028*. 2024. URL: https://www.statista.com/outlook/amo/app/worldwide.

[2] DAZO LLC. *eZaal — Спортын байгууламжийн захиалгын платформ*. Google Play, 10,000+ татагдсан, хувилбар 2.2.1, 2025-06-01. 2025. URL: https://play.google.com/store/apps/details?id=mn.e.zaal.

[3] A. Kassim, M. Rahman **and** Z. Hasan. "Development of Sports Facility Booking Mobile Application Using Flutter and Firebase". **in***International Journal of Advanced Computer Science and Applications*: 13.4 (2022), **pages** 412–420.

[4] Амарсайханы Билгүүн. "Ухаалаг гар утасны аппликейшн хөгжүүлэлт". Бакалаврын төгсөлтийн ажил. ШУТИС-МХТС, 2022.

[5] Т.Амартүвшин **and** Б.Зоригтбаатар. *Мобайл аппликейшн хөгжүүлэлт*. Улаанбаатар: ШУТИС, Хэвлэлийн газар, 2020.

[6] Google LLC. *Flutter — Build apps for any screen*. 2024. URL: https://flutter.dev/docs.

[7] Google LLC. *Material Design 3*. 2024. URL: https://m3.material.io.

[8] Google LLC. *Dart Programming Language*. 2024. URL: https://dart.dev/guides.

[9] Google LLC. *Firebase — App Development Platform*. 2024. URL: https://firebase.google.com/docs.

[10] Google LLC. *Cloud Firestore — Firebase*. 2024. URL: https://firebase.google.com/docs/firestore.

[11] Google LLC. *Authenticate Users — Firebase Authentication*. 2024. URL: https://firebase.google.com/docs/auth.

[12] Michael Bui. *flutter_local_notifications — A cross platform plugin for displaying local notifications*. 2024. URL: https://pub.dev/packages/flutter_local_notifications.

[13] Anthropic. *Claude API Documentation — Anthropic*. 2024. URL: https://docs.anthropic.com.

[14] Groq Inc. *Groq API Documentation — Fast AI Inference*. 2024. URL: https://console.groq.com/docs.

[15] OpenJS Foundation. *Node.js — JavaScript runtime built on Chrome's V8 engine*. 2024. URL: https://nodejs.org/en/docs.

[16] OpenJS Foundation. *Express — Fast, unopinionated, minimalist web framework for Node.js*. 2024. URL: https://expressjs.com.

[17] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures.* 2000. URL: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm.

[18] Docker Inc. *Docker Documentation — Develop faster. Run anywhere.* 2024. URL: https://docs.docker.com.

[19] Amazon Web Services. *Amazon EC2 — Secure and resizable compute capacity in the cloud.* 2024. URL: https://aws.amazon.com/ec2.

[20] Amazon Web Services. *Amazon Elastic Container Registry — Share and deploy container software.* 2024. URL: https://aws.amazon.com/ecr.

[21] GitHub Inc. *GitHub Actions — Automate your workflow from idea to production.* 2024. URL: https://docs.github.com/en/actions.